SIMATS ENGINEERING

Department of Artificial Intelligence And Machine Learning (AIML)

ITA0507 – Computer Vision

ASSIGNMENT 1 – CO1

CO1: Apply the fundamental concepts, image formation models, and processing techniques for analyzing Computer Vision systems.

Student Name	P.Abdul	**Register Number**	192525426
Department / Section	AIML	**Academic Year / Semester**	2025–2026
Faculty Name	Murali P	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	30.10.2026	**Date of Evaluation**	07.11.2026

Assignment Title

Apply, Compare, and Justify Image Acquisition, Formation, and Processing Techniques for a Vision-Based E-Waste Battery Sorting Line (Total Marks – 50)

INTRODUCTION

Computer Vision is an important branch of Artificial Intelligence that enables computers to understand and interpret visual information obtained from images and videos. It combines digital image processing, mathematical operations, pattern recognition and machine learning to identify objects, measure dimensions and detect defects.

In modern industries, computer vision is widely used in manufacturing, healthcare, robotics, transportation, quality inspection and automated sorting systems. It reduces the dependency on manual inspection and improves consistency, processing speed and operational safety.

This assignment focuses on the development of a vision-based e-waste battery sorting system for GreenCycle Robotics Corp. The existing facility uses human operators to sort different types of batteries. However, manual inspection creates problems such as eye fatigue, incorrect identification, missed defects and difficulty reading small chemical markings.

The proposed computer vision system is designed to identify three battery form factors: cylindrical 18650 cells, cylindrical 21700 cells and prismatic pouch cells. It must also recognize chemical codes such as LCO, LFP, NMC and NiMH. In addition, it must detect dangerous defects including rust spots, dents, cracks and pouch-cell swelling.

The system uses an industrial camera, suitable illumination, image-processing algorithms, feature extraction techniques and classification methods. The final decision is transmitted to a pneumatic piston that automatically removes unsafe or incorrectly classified batteries from the conveyor.

The main objective is to develop a reliable imaging front-end before implementing advanced deep-learning models. The assignment evaluates different techniques mathematically and selects suitable methods based on speed, accuracy, cost, reliability and industrial requirements.

SCENARIO – VISION-BASED E-WASTE BATTERY SORTING LINE

GreenCycle Robotics Corp. operates an electronic waste recycling facility where mixed consumer batteries move along a conveyor belt. At present, workers manually inspect and separate the batteries according to their physical structure and chemical composition.

Two previous sorting batches were rejected by the chemical smelter because lithium-based cells were incorrectly mixed with nickel-metal hydride (NiMH) batteries. This situation creates a serious safety risk because different battery chemistries require different recycling and handling procedures.

To solve this problem, the company proposes an automated computer vision system.

Main objectives

1. Identify cylindrical 18650, cylindrical 21700 and prismatic pouch batteries.

2. Read laser-etched chemical codes such as LCO, LFP, NMC and NiMH.
3. Detect rust spots, severe dents, cracks and pouch swelling.
4. Automatically trigger a pneumatic piston to reject unsafe batteries.
5. Store visual evidence of every processed battery for five days.
6. Maintain a maximum misclassification or defect miss rate of 1.5%.

Operating parameters

Parameter	**Specification**
Conveyor speed	1.2 m/s
Field of View	320 × 240 mm
Maximum throughput	15 cells/second
Camera working distance	400 mm
Maximum battery height	30 mm
Minimum defect size	0.3 mm
Laser text stroke	0.4 mm
Swelling measurement precision	±0.5 mm
Ambient illumination	1,500–45,000 lux
Illumination flicker	100 Hz
Edge computer	4 CPU cores, 16 GB RAM
Programming tools	Python, OpenCV and NumPy
Storage	1.5 TB SSD
Hardware budget	₹1,25,000
Archive duration	5 days
Maximum error rate	1.5%

These parameters form the engineering basis for the following ten tasks.

TASK 1 – SYSTEMS COMPARISON: MANUAL, ASSISTED AND AUTOMATED SORTING

1.1 Introduction

Computer vision systems are used to automatically obtain, process and interpret images. In industrial applications, they help identify products, inspect surface quality, measure dimensions and control manufacturing operations.

In the GreenCycle Robotics battery sorting facility, three different approaches can be considered:

- Technique A – Manual inspection.
- Technique B – Image-processing-assisted inspection.
- Technique C – Fully automated computer vision.

Each technique has different requirements in terms of cost, speed, accuracy, repeatability and safety.

The main objective is to identify which technique can satisfy the production requirement of 15 battery cells per second while maintaining a maximum error rate of 1.5%.

1.2 Technique A – Manual Inspection

Manual inspection is the traditional method in which trained workers observe batteries moving along the conveyor and classify them according to their physical appearance.

The operator visually examines the battery shape, reads its chemical markings and checks for visible defects.

Working principle

1. Mixed batteries arrive on the conveyor belt.
2. Workers visually observe each battery.
3. The battery form factor is identified.
4. Chemical markings are read manually.
5. Surface defects are checked.
6. The worker separates the battery into the appropriate category.

Advantages

- Low initial investment.
- No complex camera or software installation.
- Human operators can identify unusual defects using experience.
- Suitable for small-scale sorting operations.

- Flexible when new battery designs are introduced.

Disadvantages

- Human eye fatigue reduces inspection quality.
- Performance depends on operator experience.
- Changing factory illumination affects visibility.
- Small laser-etched chemical codes are difficult to read.
- Reflective battery surfaces create glare.
- Manual inspection cannot reliably maintain a 15-cell-per-second production rate.
- Maintaining complete inspection records is difficult.

Processing speed

The production line requires:

Throughput = 15 cells/second.

Time available for each cell:

Time = 1 / 15

Time = 0.0667 seconds.

Therefore, only approximately 66.7 milliseconds are available between successive cells.

A human operator cannot reliably complete detailed inspection of shape, chemistry and defects within this interval continuously.

Repeatability

Repeatability refers to the ability to obtain the same inspection result when the same battery is examined repeatedly.

Manual inspection may produce different results because of:

- Operator concentration.
- Experience.
- Lighting conditions.
- Fatigue.
- Viewing angle.

Two operators may also disagree about whether a small rust spot or dent is acceptable.

Therefore, manual inspection has limited repeatability.

1.3 Technique B – Image-Processing-Assisted Sorting

Image-processing-assisted sorting uses a high-resolution camera to capture battery images. The images are enhanced and displayed on an operator screen.

The final sorting decision is still made by a human operator.

Working principle

1. A high-resolution camera captures battery images.
2. The captured image is transferred to a computer.
3. Image-processing algorithms improve brightness and contrast.
4. Digital zoom highlights small chemical markings and defects.
5. The processed image is displayed on a monitor.
6. The operator examines the image and decides the battery category.
7. The operator sends the sorting command.

Image enhancement techniques

1. Contrast enhancement

Contrast enhancement improves the difference between dark and bright regions.

It helps reveal small rust spots and etched characters.

2. Digital zoom

Digital zoom enlarges a selected image region.

It helps operators inspect small chemical codes such as LCO, LFP, NMC and NiMH.

However, digital zoom cannot recover image details that were never captured by the camera sensor.

3. Noise reduction

Gaussian filtering can reduce unwanted noise.

It improves image appearance and makes further inspection easier.

Advantages

- Better visibility than direct manual inspection.
- Small defects can be enlarged.
- Images can be stored for future verification.
- Reduces some visual effort for operators.

- Provides a digital inspection record.

Disadvantages

- Human decision-making is still required.
- Display interpretation introduces additional delay.
- Digital zoom does not increase actual optical resolution.
- Operator fatigue can still affect the result.
- Difficult to maintain continuous 15-cell-per-second operation.

Repeatability

Image enhancement provides a consistent visual representation. However, the final decision still depends on the operator.

Therefore, repeatability is better than direct manual inspection but lower than a properly validated automated system.

1.4 Technique C – Fully Automated Computer Vision

Fully automated computer vision uses cameras, image-processing algorithms, classification methods and hardware controllers to perform the entire sorting operation.

The system captures an image, identifies the battery, detects defects and sends the required signal to the pneumatic rejection mechanism.

Working principle

1. Batteries move along the conveyor.
2. An industrial camera captures images using controlled illumination.
3. Image preprocessing removes noise and improves contrast.
4. Segmentation separates the battery from the conveyor background.
5. Feature extraction measures shape, size and surface characteristics.
6. OCR identifies the laser-etched chemical markings.
7. Classification algorithms identify the battery type.
8. Defect detection identifies rust, dents, cracks and swelling.
9. A decision module determines whether the battery should be accepted or rejected.
10. A controller activates the pneumatic piston at the correct position.

Main components

Industrial camera: Captures high-resolution images of moving batteries.

LED illumination: Provides stable and uniform lighting.

Edge computer: Runs Python, OpenCV and NumPy algorithms.

Classification module: Identifies battery form factors and chemistry categories.

Defect detection module: Detects structural damage.

Pneumatic piston: Removes rejected batteries from the conveyor.

Storage unit: Stores images and inspection information for five days.

Advantages

- High-speed inspection.
- Consistent decision-making.
- Reduced human dependency.
- Continuous operation.
- Automatic image archiving.
- Better production traceability.
- Suitable for industrial automation.

Disadvantages

- Higher initial investment.
- Requires camera calibration.
- Needs software development and maintenance.
- Incorrect training data can affect classification.
- Hardware or communication failures must be handled safely.

1.5 Comparison of Three Techniques

Evaluation factor	**Manual**	**Assisted**	**Automated**
Inspection	Human eyes	Camera + human	Camera + software
Speed	Low	Medium	High
Repeatability	Low	Medium	High when validated
Objectivity	Operator-dependent	Partially improved	Consistent algorithm
Continuous operation	Limited	Limited	Suitable

Image storage	Difficult	Possible	Automatic
Initial cost	Low	Medium	High
Maintenance	Human training	Camera and display	Hardware and software
Safety control	Manual	Human command	Automatic reject

1.6 Repeatability and Cohen's Kappa

Cohen's Kappa is a statistical measure used to evaluate agreement between two observers after removing the agreement that could occur by chance.

It is calculated using:

$\kappa = (Po - Pe) / (1 - Pe)$

Where:

- Po = Observed agreement.
- Pe = Expected agreement by chance.
- κ = Cohen's Kappa value.

Interpretation

Kappa value Interpretation

Below 0 Poor agreement 0.00–

0.20 Slight agreement

0.21–0.40 Fair agreement

0.41–0.60 Moderate agreement

0.61–0.80 Substantial agreement

0.81–1.00 Almost perfect agreement

For manual inspection, two operators may classify the same battery differently. Cohen's Kappa can measure their agreement.

For an automated system, the more appropriate evaluation includes confusion matrices, precision, recall and defect miss rate against verified ground-truth labels.

Actual Kappa values cannot be calculated without observations from two inspectors.

1.7 Objectivity and Inter-Rater Variance

Objectivity means that an inspection result should not change unnecessarily depending on the person performing the inspection.

Manual inspection has higher inter-rater variation because different operators may have different experience and judgement.

Assisted inspection reduces some variation because all operators see enhanced images.

Automated inspection uses the same algorithm and decision thresholds for every battery. This improves consistency, although the system can still make systematic errors if its calibration or model is incorrect.

1.8 Non-Contact Safety and Continuous Endurance

The proposed computer vision system inspects batteries without physically touching them.

This is important because damaged batteries may contain hazardous materials or may become unstable during handling.

A camera-based system also reduces the need for workers to remain close to moving batteries.

For continuous operation, the automated system can work throughout production shifts, provided that camera temperature, illumination, storage, communication and mechanical components are monitored.

1.9 Audit Traceability and Lifecycle Cost

Traceability means maintaining records of every processed battery.

The automated system should store:

- Unique battery identification number.
- Date and time of inspection.
- Original captured image.
- Detected battery category.
- Recognized chemistry code.
- Defect information.
- Classification confidence.
- Accept or reject decision.
- Pneumatic actuator command status.

The five-day archive requirement makes automatic storage necessary.

Manual inspection has low equipment cost but higher ongoing labour cost. Assisted inspection adds cameras and display systems. Fully automated inspection has a higher initial cost but can reduce repeated manual inspection and improve production consistency.

A complete lifecycle-cost analysis should include equipment purchase, installation, maintenance, electricity, software updates, operator supervision and replacement parts.

1.10 Difference Between Image Processing, Computer Vision, Machine Vision and Computer Graphics

Image Processing

Image processing performs operations directly on images.

Examples:

- Noise removal.
- Brightness adjustment.
- Contrast enhancement.
- Colour conversion.
- Image filtering.
- Thresholding.

Input: Original battery image.

Output: Enhanced or transformed battery image.

Computer Vision

Computer vision interprets the information present in an image.

Examples:

- Identifying an 18650 battery.
- Recognizing LFP or NiMH text.
- Detecting rust spots.
- Measuring pouch swelling.

Input: Image or video.

Output: Meaningful information about the object.

Machine Vision

Machine vision combines cameras, illumination, software and industrial control hardware to perform an automated task.

In this assignment, machine vision captures battery images, identifies their categories and controls the pneumatic rejection mechanism.

Input: Physical battery and image sensor data.

Output: Industrial sorting decision and actuator signal.

Computer Graphics

Computer graphics generates images using computer models and graphical techniques.

Examples include:

- 3D battery models.
- Conveyor layout simulation.
- Industrial machine illustrations.
- Graphical user interfaces.

Input: Models, geometry and scene descriptions.

Output: Computer-generated images.

1.11 Overall Workflow Diagram

Battery arrival

↓

Industrial camera captures image

↓

Image preprocessing and enhancement

↓

Battery segmentation

↓

Shape and feature extraction

↓

Chemistry-code recognition

↓

Defect detection

↓

Decision-making module

↓

Accept / Reject classification

↓

Pneumatic piston control

↓

Image and inspection record storage

1.12 Final Engineering Decision

Technique C, fully automated computer vision, is the most suitable approach for the GreenCycle Robotics battery sorting line.

The main reason is the production requirement of 15 cells per second. Manual and assisted methods depend on continuous human observation and decision-making, which makes it difficult to achieve the required speed and repeatability.

Automated computer vision provides consistent image acquisition, chemical-code recognition, defect detection, automatic rejection and five-day image archiving.

Technique B can still be used as a supervisory interface so that operators can review uncertain cases and monitor the production line.

The automated system must be validated using representative batteries, different lighting conditions, surface defects and actual production speeds. The maximum permitted defect miss or misclassification rate is 1.5%, and this value must be demonstrated through testing rather than assumed.

Conclusion: A fully automated computer vision system supported by controlled illumination, calibrated cameras and reliable industrial control is recommended for the battery sorting facility.

TASK 2 – SYSTEM ARCHITECTURE: PIPELINE LEVEL DEPTH COMPARISON

2.1 Introduction

A computer vision system processes digital images to extract useful information about objects, their shapes, colours, positions and physical conditions. In the GreenCycle Robotics battery sorting facility, the vision system must identify different battery types, read their chemical labels and detect physical defects.

The system receives images from an industrial camera installed above a moving conveyor. These images are processed through different software levels before the final sorting decision is sent to a pneumatic rejection mechanism.

The depth of a computer vision pipeline determines how much information is extracted from an image. A simple pipeline may examine individual pixel values, while a more advanced pipeline can identify object boundaries, calculate geometric features and classify objects using machine learning.

Three software architecture techniques are considered:

1. Technique A – Low-Level Image Processing Only.
2. Technique B – Low-Level and Mid-Level Image Processing.
3. Technique C – Low-Level, Mid-Level and High-Level Processing with Classical Machine Learning.

The objective is to compare their processing capabilities, accuracy, computational requirements and suitability for the battery sorting line.

2.2 Need for Different Processing Levels

A single pixel value does not always provide sufficient information to identify a battery or detect a defect. For example, a dark region in an image may represent rust, a shadow, an oil stain or a natural difference in surface reflection.

Therefore, the system should progressively transform raw image data into meaningful information.

The three main processing levels are:

- **Low-level processing:** Improves image quality and performs basic pixel operations.
- **Mid-level processing:** Extracts boundaries, regions, shapes and geometric properties.
- **High-level processing:** Uses extracted features to classify objects and make decisions.

These levels work together to improve the reliability of the sorting system.

2.3 Technique A – Low-Level Image Processing Only

2.3.1 Definition

Low-level image processing operates directly on the pixels of a digital image. It does not require an understanding of the complete object or its category.

The main purpose is to improve image quality and identify abnormal pixel patterns.

In the battery sorting application, low-level processing can be used to detect dark areas, sudden intensity changes and possible surface defects.

2.3.2 Working Principle

The input image is captured by the industrial camera. It is then converted into grayscale to simplify the processing operation.

A Gaussian blur operation reduces small image noise. Adaptive thresholding converts the grayscale image into a binary image. The system then counts dark pixels or pixels outside an expected intensity range.

If the number of abnormal pixels exceeds a predefined threshold, the system flags the battery for further inspection.

2.3.3 Gaussian Blur

Gaussian blur is used to reduce unwanted image noise before thresholding.

The Gaussian filter uses a weighted average of neighbouring pixels. Pixels closer to the centre receive higher weights.

The two-dimensional Gaussian function is:

$$G(x,y) = 1/(2\pi\sigma^2) \times \exp(-(x^2+y^2)/(2\sigma^2))$$

Where:

- G(x,y) represents the Gaussian weight.
- x and y represent pixel positions.
- σ represents the standard deviation.
- π is a mathematical constant.

A larger σ produces stronger smoothing, while a smaller σ preserves more fine details.

For the battery sorting system, excessive smoothing must be avoided because the smallest defect is only 0.3 mm. A strong blur may remove small rust spots or micro-cracks.

2.3.4 Adaptive Thresholding

Thresholding converts a grayscale image into a binary image.

A simple global threshold uses one fixed threshold value for the entire image. However, factory illumination changes throughout the day. Adaptive thresholding calculates a local threshold for different image regions.

The binary output can be represented as:

$B(x,y) = 255$, if $I(x,y) > T(x,y)$

$B(x,y) = 0$, otherwise.

Where:

- $B(x,y)$ is the output binary pixel.
- $I(x,y)$ is the original grayscale pixel.
- $T(x,y)$ is the locally calculated threshold.

Adaptive thresholding can be useful when different parts of an image have different brightness levels.

2.3.5 Dark Pixel Counting

After thresholding, the software counts pixels that are classified as dark or abnormal.

Let:

N = total number of pixels in the inspection region.

D = number of dark pixels.

The dark-pixel ratio is:

$$\text{Dark Pixel Ratio} = D/N$$

If the ratio is greater than a selected threshold, the system produces a defect warning.

For example, suppose an inspection region contains 10,000 pixels and 250 pixels are classified as abnormal.

$$\text{Dark Pixel Ratio} = 250/10{,}000 = 0.025 = 2.5\%$$

This value can be compared with a threshold established during calibration.

This example demonstrates the calculation only. The actual threshold must be established using representative battery images.

2.3.6 Advantages

1. Simple implementation using OpenCV.
2. Low computational requirements.
3. Fast execution on a CPU.
4. Easy to understand and debug.
5. Suitable for basic image enhancement.

6. Does not require a machine learning training dataset.

2.3.7 Limitations

1. It cannot reliably understand complete object shapes.
2. Shadows may be incorrectly classified as defects.
3. Oil stains may produce false alarms.
4. Touching batteries may be treated as one region.
5. Different battery materials may produce different pixel intensities.
6. Fixed thresholds may fail under changing lighting.
7. It cannot independently identify all battery categories.

2.3.8 Shadow Boundary False Triggers

A major problem with low-level processing is that shadows can produce intensity changes similar to defects.

For example, a battery moving below an overhead light may produce a dark shadow along one side. A simple dark-pixel detector may interpret this shadow as rust or a surface crack.

This creates a false positive.

A false positive occurs when the system identifies a defect even though the inspected battery is healthy.

A false negative occurs when the system fails to identify an actual defect.

Both errors are important because the plant has a maximum permitted misclassification or defect miss rate of 1.5%.

Therefore, low-level pixel counting should not be used as the only final safety decision method.

2.4 Technique B – Low-Level and Mid-Level Image Processing

2.4.1 Definition

Mid-level image processing adds object and shape analysis to the basic pixel operations.

Instead of only examining individual pixels, the system identifies connected regions, object boundaries and geometric properties.

This approach is more informative than low-level processing because the system can analyse the shape of a battery.

2.4.2 Working Principle

The processing sequence is:

1. Capture the battery image.
2. Convert the image into grayscale.
3. Apply Gaussian blur.
4. Apply thresholding or edge detection.
5. Identify object contours.
6. Calculate bounding rectangles.
7. Extract geometric features.
8. Apply predefined expert-system rules.
9. Generate a classification or defect warning.

2.4.3 Contour Detection

A contour is a curve joining the boundary points of an object having the same intensity or binary classification.

OpenCV provides the cv2.findContours() function to identify contours in a binary image.

Contours can be used to estimate:

- Object boundaries.
- Object area.
- Object perimeter.
- Object dimensions.
- Number of detected objects.

For battery sorting, contours help separate cylindrical cells from prismatic pouch cells when the objects are sufficiently separated in the image.

2.4.4 Minimum Area Rectangle

The cv2.minAreaRect() function calculates a rotated rectangle enclosing a contour with minimum area.

The rectangle provides:

- Centre position.
- Width.
- Height.
- Rotation angle.

The aspect ratio can be calculated as:

Aspect Ratio = Longer Side / Shorter Side

This feature can help distinguish objects with different shapes.

However, the aspect ratio alone cannot reliably distinguish every 18650 and 21700 battery. Their dimensions and image orientation may overlap under certain viewing conditions. Additional features or measurements are required.

2.4.5 Image Moments

Image moments describe the distribution of pixels within an object region.

The spatial moment is:

$M(p,q) = \Sigma x \, \Sigma y \, x^p y^\varphi \, I(x,y)$

Where:

- p and q are moment orders.
- x and y represent pixel coordinates.
- I(x,y) represents the binary or grayscale image intensity.

The zeroth-order moment represents area for a binary object.

The first-order moments can be used to calculate the object centroid:

$Cx = M(1,0)/M(0,0)$

$Cy = M(0,1)/M(0,0)$

The centroid provides the approximate centre of the detected battery.

2.4.6 Contour Circularity

Circularity measures how closely a contour resembles a circle.

$Circularity = 4\pi A/P^2$

Where:

- A is contour area.
- P is contour perimeter.

A perfect circle has a circularity value close to 1. Irregular shapes generally have lower values.

Circularity can support the identification of cylindrical battery end faces. It should not be treated as a complete classifier because perspective, occlusion and image orientation can change the measured value.

2.4.7 Expert-System Rules

An expert system applies predefined rules to extracted features.

Example rules:

- If the contour is approximately circular and its dimensions match a cylindrical cell, classify it as a possible cylindrical battery.
- If the contour has a rectangular shape and elongated dimensions, classify it as a possible prismatic cell.
- If the contour contains an irregular protrusion, flag it for swelling inspection.
- If a surface region has abnormal texture and colour, mark it for possible rust inspection.

These are preliminary rules. Actual limits must be calibrated using the plant's battery samples.

2.4.8 Advantages

1. Provides more information than individual pixel analysis.
2. Can identify object boundaries.
3. Supports geometric classification.
4. Can estimate object position and orientation.
5. Uses relatively lightweight OpenCV operations.
6. Can run on a CPU without a dedicated GPU.
7. Provides interpretable rules for engineering inspection.

2.4.9 Limitations

1. Touching objects may merge into one contour.
2. Strong shadows can distort object boundaries.
3. Oil smudges can create additional contours.
4. Irregularly positioned batteries may produce inaccurate geometric features.
5. Fixed rules may not cover every battery condition.
6. Performance depends on image quality and segmentation.
7. New battery types may require manually designed rules.

2.5 Technique C – Low-Level, Mid-Level and High-Level Processing with Machine Learning

2.5.1 Definition

High-level processing uses the results of low-level and mid-level operations to make meaningful decisions.

In this architecture, image processing extracts the important features, and a machine learning classifier uses these features to predict the battery category or condition.

The assignment proposes classical machine learning algorithms such as:

- Support Vector Machine (SVM).
- k-Nearest Neighbours (k-NN).

The selected algorithm receives a feature vector and produces a predicted category.

2.5.2 Feature Vector Extraction

A feature vector is a collection of numerical values describing an object.

A battery feature vector may contain:

X = [Area, Perimeter, Aspect Ratio, Circularity, Width, Height, Mean Intensity, Texture Features]

Each detected battery is represented using these numerical features.

The features are supplied to the machine learning classifier.

2.5.3 Support Vector Machine

Support Vector Machine is a supervised machine learning algorithm used for classification and regression.

For classification, SVM attempts to find a decision boundary that separates different categories.

For a linear classifier, the decision function can be represented as:

$f(x) = w^T x + b$

Where:

- x is the input feature vector.
- w is the weight vector.
- b is the bias term.

The predicted class depends on which side of the decision boundary the feature vector lies.

SVM can also use kernel functions to model non-linear boundaries.

Possible battery categories include:

- 18650 cylindrical battery.
- 21700 cylindrical battery.
- Prismatic pouch battery.

- Healthy battery.
- Suspected defective battery.

The actual categories should be defined separately for form-factor classification and defect classification.

2.5.4 k-Nearest Neighbours

k-Nearest Neighbours is a supervised learning algorithm that classifies a new sample based on the labels of its nearest training samples.

The Euclidean distance between two feature vectors is:

$$d(x,y) = \sqrt{\Sigma_i}(x_i - y_i)^2$$

The algorithm identifies the k closest training samples.

The majority class among those neighbours is assigned to the new sample.

For example, if k = 5 and three of the five nearest training samples belong to the 18650 category, the new sample may be classified as an 18650 battery.

The value of k must be selected through validation. Feature scaling is also important because area, perimeter and intensity may have very different numerical ranges.

2.5.5 Training and Testing

The machine learning model requires labelled examples.

The dataset should include:

- Images of 18650 batteries.
- Images of 21700 batteries.
- Images of prismatic pouch cells.
- Healthy battery images.
- Rust-affected battery images.
- Dented battery images.
- Swollen pouch-cell images.
- Images captured under different lighting conditions.
- Images containing touching batteries and partial occlusions.

The dataset should be divided into training, validation and test sets.

The training set is used to learn the classifier. The validation set helps select parameters. The test set is used to evaluate performance on previously unseen examples.

Images from the same physical battery should not be randomly distributed across training and test sets in a way that creates near-duplicate leakage. Where possible, separate physical battery samples should be used for independent evaluation.

2.5.6 Advantages

1. Uses object geometry and learned classification boundaries.
2. Can handle more variation than fixed rules when trained appropriately.
3. Provides a structured way to classify battery categories.
4. Supports separate classification and defect-detection models.
5. Can operate on extracted features without requiring a large deep-learning network.
6. SVM and k-NN can be implemented using CPU-compatible libraries.
7. Feature contributions can be inspected more easily than in many end-to-end deep-learning systems.

2.5.7 Limitations

1. Requires labelled training data.
2. Poor feature extraction reduces classification performance.
3. SVM parameter selection affects results.
4. k-NN prediction can become slower as the training dataset grows.
5. New battery types may require retraining.
6. Feature vectors may not capture complex visual patterns.
7. Performance must be evaluated under the actual operating conditions.

2.6 Comparison of the Three Techniques

Comparison factor	**Technique A**	**Technique B**	**Technique C**
Processing levels	Low	Low + Mid	Low + Mid + High
Pixel enhancement	Yes	Yes	Yes
Contour extraction	No	Yes	Yes
Geometric features	Limited	Yes	Yes
Machine learning	No	No	Yes

Object classification	Very limited	Rule-based	Learned classifier
CPU requirement	Low	Low to moderate	Depends on feature count and classifier
Interpretability	High	High	Moderate to high
Training dataset	Not required	Not required for fixed rules	Required
Complex variation handling	Weak	Moderate	Potentially stronger
Main limitation	Pixel ambiguity	Rule limitations	Data and feature dependence

2.7 Accuracy and Computational Latency Evaluation

The assignment requires a comparison of grading accuracy and computational latency for challenging test cases, including touching batteries, oil smudges and strong shadows.

However, the assignment brief does not provide a labelled image dataset or actual experimental measurements. Therefore, exact accuracy and latency values cannot be honestly reported as measured results.

The following table should be completed after running all three configurations on the same test dataset.

Test condition	**Technique A accuracy**	**Technique B accuracy**	**Technique C accuracy**
Separate clean batteries	To be measured	To be measured	To be measured
Side-by-side touching batteries	To be measured	To be measured	To be measured
Oil smudges	To be measured	To be measured	To be measured
Strong shadows	To be measured	To be measured	To be measured
Reflective surfaces	To be measured	To be measured	To be measured
Overall test accuracy	To be measured	To be measured	To be measured

The latency table should be recorded separately.

Configuration	Average latency (ms)	Maximum latency (ms)	CPU usage
Technique A	To be measured	To be measured	To be measured
Technique B	To be measured	To be measured	To be measured
Technique C	To be measured	To be measured	To be measured

2.7.1 Accuracy Formula

Accuracy is calculated as:

Accuracy = (Correct Predictions / Total Predictions) × 100

For a classification problem:

Accuracy = (TP + TN)/(TP + TN + FP + FN) × 100

Where:

- TP = True Positives.
- TN = True Negatives.
- FP = False Positives.
- FN = False Negatives.

For the battery safety application, accuracy alone is insufficient. The system must also measure defect recall because missing a dangerous swollen or damaged cell may have serious consequences.

2.7.2 Processing Latency

Processing latency is the time required to process one image and produce a decision.

Latency can be measured using:

Latency = Decision Completion Time − Image Acquisition Completion Time

For a valid comparison, all three techniques must use the same image resolution, test images, computer hardware and measurement procedure.

The assignment specifies a 4-core CPU and no GPU. Therefore, the experiment should be performed on the specified CPU environment rather than using a different high-performance computer.

2.8 Comparison with End-to-End Deep Learning

Deep learning is another approach to computer vision. It differs from the classical feature-based pipeline because a neural network can learn useful image representations directly from training images.

2.8.1 Convolutional Neural Networks

A Convolutional Neural Network (CNN) applies convolution filters to image data.

The network learns patterns such as:

- Edges.
- Corners.
- Textures.
- Surface marks.
- Object parts.
- Higher-level object features.

Early layers often respond to simple patterns, while deeper layers combine these patterns into more complex representations.

2.8.2 Vision Transformer

A Vision Transformer (ViT) divides an image into patches and processes their representations using transformer attention mechanisms.

It can learn relationships between distant image regions.

However, a ViT may require substantial training data and computational resources depending on the model size and deployment method.

2.8.3 Classical Machine Learning vs Deep Learning

Factor	**Classical ML pipeline**	**CNN / ViT**
Feature extraction	Designed using image processing	Learned from data
Input	Numerical feature vectors	Image pixels or patches
Training data	Labelled feature samples	Usually many labelled images, depending on task
CPU deployment	Often practical for small models	Model-dependent

Interpretability	Feature values and rules can be inspected	More difficult to interpret
Complex appearance variation	Depends on designed features	Can learn complex patterns
Deployment complexity	Generally lower	Often higher

2.8.4 Structural Data Constraints

The GreenCycle facility has a 4-core CPU, 16 GB RAM and no dedicated GPU.

This means the selected architecture must consider:

- Model size.
- Memory usage.
- Number of image operations.
- Processing time.
- Number of cells per second.
- Image storage requirements.
- Reliability under continuous operation.

A deep-learning model should not be selected only because it is more advanced. It must also meet the processing deadline and operate within the available hardware resources.

2.9 Complete Level-Annotated Pipeline Block Diagram

Figure 2.1 – Three-Level Computer Vision Processing Architecture

[Camera Image Acquisition]
↓
[LOW-LEVEL PROCESSING]

- Image loading
- Grayscale conversion
- Gaussian blur
- Adaptive thresholding
- Noise reduction
  ↓
  [MID-LEVEL PROCESSING]
- Contour detection

- Object segmentation
- Bounding rectangle
- Aspect ratio
- Image moments
- Circularity
  ↓
  [HIGH-LEVEL PROCESSING]
- Feature vector creation
- SVM or k-NN classifier
- Battery category prediction
- Defect confidence estimation
  ↓
  [DECISION AND CONTROL]
- Accept / reject decision
- Record inspection result
- Send control signal
  ↓
  [PNEUMATIC REJECTION SYSTEM]

The diagram represents the proposed three-level architecture. Actual hardware control requires suitable industrial interfaces, electrical protection and timing validation.

2.10 Proposed Python and OpenCV Implementation

The following sample demonstrates the basic low-level and mid-level operations. It is an educational starting point and does not constitute a validated safety inspection system.

```
import cv2

import numpy as np


# Load battery image

image = cv2.imread("battery.jpg")


if image is None:

    raise FileNotFoundError("Image not found")
```

```
# Low-level processing
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(
    gray,
    (5, 5),
    0
)

binary = cv2.adaptiveThreshold(
    blur,
    255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV,
    21,
    5
)

# Mid-level processing
contours, hierarchy = cv2.findContours(
    binary,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:
```

```
area = cv2.contourArea(contour)

if area < 100:
    continue

perimeter = cv2.arcLength(
    contour,
    True
)

x, y, w, h = cv2.boundingRect(contour)

aspect_ratio = max(w, h) / max(1, min(w, h))

circularity = 0

if perimeter > 0:
    circularity = (
        4 * np.pi * area
    ) / (perimeter * perimeter)

print("Area:", area)
print("Perimeter:", perimeter)
print("Aspect ratio:", aspect_ratio)
print("Circularity:", circularity)

cv2.rectangle(
    image,
```

```
        (x, y),
        (x + w, y + h),
        (0, 255, 0),
        2
    )

cv2.imshow("Detected Objects", image)
cv2.imshow("Binary Image", binary)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

A separate feature-extraction and training stage would be needed to implement Technique C using an SVM or k-NN classifier. The training dataset must contain correctly labelled examples.

2.11 Engineering Selection and Justification

Technique A is useful for simple image enhancement and preliminary defect screening. However, its dependence on pixel intensity makes it vulnerable to shadows, oil stains and changing illumination.

Technique B provides additional object information through contour extraction and geometric measurements. It is more suitable for separating and analysing battery shapes, but its performance depends on segmentation quality and manually selected rules.

Technique C combines the previous processing levels with a trained classifier. It can learn decision boundaries from labelled examples and may handle appearance variations better than fixed rules when the dataset represents the actual factory conditions.

For the GreenCycle battery sorting line, Technique C is the preferred software architecture for further development, provided that it meets the CPU processing deadline and passes independent safety validation. Technique B should remain available as an interpretable feature-extraction and diagnostic layer.

The final design should not rely on a single machine learning confidence value to authorize unsafe battery handling. Critical defect cases should use conservative rejection rules and a validated fail-safe response.

2.12 Task 2 Conclusion

This task compared three levels of computer vision processing for the automated battery sorting system.

Technique A performs low-level pixel operations and is computationally simple, but it cannot reliably distinguish every defect from a shadow or surface variation.

Technique B adds contour detection and geometric feature extraction. It provides a better understanding of object shape and position but depends on manually designed rules.

Technique C adds a classical machine learning classifier to the extracted feature vectors. It provides a more flexible classification approach, although it requires labelled training data and careful validation.

A three-level pipeline is recommended as the main development architecture. Its final suitability must be demonstrated through measured accuracy, defect recall, processing latency and continuous-operation tests on the actual edge computer.

2.13 Additional Engineering Considerations

A. Dataset Preparation

The dataset should represent different battery shapes, materials, orientations and surface conditions. It should include healthy and defective samples captured under different lighting conditions.

B. Data Labelling

Every training image should have an appropriate label. Defect images should identify the defect category and, where necessary, the defect location.

C. Model Validation

The trained classifier should be evaluated on samples that were not used during training. This helps estimate its ability to classify previously unseen batteries.

D. Error Handling

The system should have a defined response for unreadable images, incomplete detections, uncertain classifications and communication failures.

E. Traceability

The system should save the inspection image, timestamp, predicted category, defect result and final sorting decision according to the plant's five-day visual evidence requirement.

F. Future Improvements

Future development may include more advanced object detection, optical character recognition for laser-etched chemistry codes, improved illumination control and thermal imaging for internal defect inspection.

2.14 Suggested Figures for Task 2

Insert the following illustrations after their corresponding explanations:

1. **Figure 2.1:** Low-level, mid-level and high-level computer vision pipeline.
2. **Figure 2.2:** Gaussian blur before-and-after image example.
3. **Figure 2.3:** Adaptive thresholding of a battery image.
4. **Figure 2.4:** Contour detection around cylindrical and pouch batteries.
5. **Figure 2.5:** Bounding rectangle and centroid measurement.
6. **Figure 2.6:** SVM classification decision boundary.
7. **Figure 2.7:** k-Nearest Neighbours classification example.
8. **Figure 2.8:** Classical machine learning compared with CNN processing.
9. **Figure 2.9:** Complete GreenCycle battery sorting software architecture.

Suggested caption: These figures illustrate how raw camera images are progressively converted into pixel information, object geometry, classification features and final sorting decisions.

TASK 3 – DIGITAL REPRESENTATION: COLOR SPACE SELECTION FOR OXIDE RUST ANALYSIS

3.1 Introduction

Digital images represent real-world objects through numerical pixel values. A computer vision system does not directly understand rust, scratches, dents, or battery chemistry. It receives an image as a collection of numerical values and processes those values to identify meaningful visual patterns.

In the GreenCycle Robotics battery-sorting system, detecting rust is an important inspection requirement. Rust can indicate corrosion and possible deterioration of the battery casing. However, the rust spots in this application have very low contrast. Their intensity difference from healthy casing material may be only 5–8 grey levels on an 8-bit scale.

The inspection environment also has changing illumination. Ambient light can vary from 1,500 lux to 45,000 lux. Therefore, the same rusted battery may produce different pixel values under different lighting conditions. A color representation that works under one lighting condition may produce unreliable results under another.

This task compares three possible color-space approaches:

1. Raw RGB/BGR representation.
2. HSV color space using Hue and Saturation/Value information.

3. CIE L*ab** color space using color difference (ΔE) from a calibrated healthy-surface reference.

The objective is to identify a suitable representation for detecting oxide rust while reducing false detections caused by changes in illumination.

3.2 Understanding Digital Image Representation

A digital image is represented as a two-dimensional or three-dimensional array of numerical values.

A grayscale image contains one intensity value for each pixel. A color image generally contains three values per pixel, representing its color channels.

For an 8-bit image, each channel can represent values from 0 to 255.

- 0 represents the minimum channel intensity.
- 255 represents the maximum channel intensity.
- Intermediate values represent different intensity levels.

A color image with three channels stores three values for every pixel. These values are used to represent the color information at that location.

For example, an image pixel may contain the values:

B = 80, G = 100, R = 120

This represents one color pixel in the BGR channel order used by OpenCV.

3.2.1 Image dimensions and array structure

A color image can be represented as:

image.shape = (height, width, channels)

For example:

(480, 640, 3)

This indicates:

- Height = 480 pixels.
- Width = 640 pixels.
- Number of channels = 3.

The total number of stored channel values is:

480 × 640 × 3 = 921,600 values

If each channel value uses one byte, the uncompressed image requires approximately 921,600 bytes, excluding file headers and other storage overhead.

3.2.2 Row, column and channel mapping

In NumPy and OpenCV, an image pixel is accessed using:

image[y, x]

Here:

- y represents the row position.
- x represents the column position.
- The optional third index selects a color channel.

For example:

image[100, 200]

returns the three channel values at row 100 and column 200.

The expression:

image[100, 200, 0]

accesses the blue channel in an OpenCV BGR image.

It is important to remember that image coordinates are normally written as (x, y), but NumPy array indexing uses [y, x].

3.3 Technique A – Raw RGB/BGR Color Representation

3.3.1 Working principle

RGB is a color representation based on three primary color channels:

- Red (R)
- Green (G)
- Blue (B)

Different combinations of these three values produce different colors.

OpenCV normally loads standard color images in BGR order rather than RGB order. Therefore, the channel arrangement is:

BGR = (Blue, Green, Red)

A raw BGR image can be inspected by comparing the three channel values of each pixel.

For the battery inspection system, the algorithm can compare the color values of a suspected rust region with a reference region representing healthy casing material.

3.3.2 Example of rust detection using BGR values

Suppose a healthy metal surface has the following illustrative pixel value:

Healthy surface = (120, 125, 130)

A suspected rust pixel may have:

Suspected rust = (90, 105, 125)

The difference between these values can help identify a possible color change.

However, these values are only an example. Actual values must be obtained from images captured by the GreenCycle inspection camera.

A simple color-distance calculation can be written as:

$D = \sqrt{[(B1-B2)^2 + (G1-G2)^2 + (R1-R2)^2]}$

A large distance indicates that the two pixels have different channel values.

The system can compare the calculated distance with a threshold to identify possible rust regions.

3.3.3 Advantages of raw BGR representation

1. It is directly available after reading an image using OpenCV.
2. It does not require an additional color-space conversion.
3. It is simple to understand and implement.
4. It preserves the original channel values.
5. It can be useful when illumination and camera settings are controlled.
6. It supports basic image operations and color comparisons.

3.3.4 Limitations of raw BGR representation

The main limitation is its sensitivity to illumination.

When the amount of light changes, the intensity of the red, green and blue channels may also change. Therefore, the same physical surface may produce different pixel values under different conditions.

For example, a rust spot captured under bright sunlight may have higher channel values than the same rust spot captured under dim lighting.

This creates difficulties when a fixed threshold is used.

Other limitations include:

- Shadows can make healthy material appear darker.

- Reflections can make rust regions appear brighter.
- Camera exposure changes can affect all three channels.
- White-balance changes can shift channel relationships.
- A fixed BGR threshold may not work across all operating conditions.

3.3.5 Suitability for GreenCycle Robotics

Raw BGR is useful for initial image capture and basic preprocessing. However, using only raw BGR thresholds is not sufficiently reliable for the GreenCycle environment because the ambient illumination varies significantly.

It can be retained as the original input representation, while another color space is used for more stable color analysis.

3.4 Technique B – HSV Color Space

3.4.1 Working principle

HSV represents color using three components:

- Hue (H)
- Saturation (S)
- Value (V)

Each component describes a different property of color.

Hue: Represents the basic color category, such as red, orange, yellow or green.

Saturation: Represents the strength or purity of the color. A highly saturated color appears more vivid, while a low-saturation color appears closer to grey.

Value: Represents brightness or intensity.

Unlike RGB/BGR, HSV separates color information from brightness information. This separation can make certain color-detection tasks easier.

3.4.2 HSV representation for rust detection

Rust commonly produces reddish-brown, orange-brown or brownish surface regions. The Hue component can help identify the color category, while Saturation can help distinguish colored rust from neutral metal surfaces.

The Value component provides brightness information and can help identify very dark or very bright regions.

A basic rust-detection method may apply a range of Hue and Saturation values, followed by morphological operations to remove small noise regions.

However, a single fixed HSV threshold should not be assumed to detect every rust type. Actual battery casing samples must be used to establish suitable thresholds.

3.4.3 Advantages of HSV

1. It separates hue information from brightness.
2. It is convenient for identifying colored regions.
3. It can be more useful than raw BGR for color-based segmentation.
4. It allows independent thresholding of Hue, Saturation and Value.
5. It can help reduce some brightness-related effects when the hue remains stable.
6. It is supported directly by OpenCV.

3.4.4 Limitations of HSV

HSV is not completely independent of illumination.

At very low saturation, Hue becomes unstable or less meaningful. Strong reflections can also reduce saturation and change the apparent color of a surface.

Very bright illumination can cause pixel clipping, while very dim illumination can increase sensor noise.

Therefore, HSV may work well for clearly colored rust regions but may be less reliable for subtle corrosion with very low color contrast.

3.4.5 Suitability for GreenCycle Robotics

HSV is a useful intermediate approach. It can help identify reddish-brown rust regions and separate color from brightness.

However, because the rust contrast in this assignment is very low, HSV should be tested carefully under different illumination levels. It should not be treated as a complete solution to illumination variation.

3.5 Technique C – CIE L*a*b* Color Space

3.5.1 Working principle

CIE L*a*b*, commonly called Lab, is a color representation designed to describe color differences in a way that is more closely related to human visual perception than raw RGB values.

It contains three components:

- L*: Lightness.
- a*: Green-to-red color axis.
- b*: Blue-to-yellow color axis.

The L* component represents lightness. The a* component represents the position between green and red. The b* component represents the position between blue and yellow.

For rust analysis, the a* and b* components are particularly useful because oxide rust can produce a change towards reddish and yellowish color directions.

The L* component can also be used to examine differences in surface brightness.

3.5.2 Why Lab is useful for rust analysis

A healthy metal casing and a rusted casing may have different color characteristics even when their brightness values are similar.

By converting the image into Lab, the system can compare the color of each pixel with a reference color obtained from a healthy battery casing.

The difference between the reference and observed color can be calculated using ΔE.

A simple method is the CIE76 Euclidean color difference:

$$\Delta E76 = \sqrt{[(L1-L2)^2 + (a1-a2)^2 + (b1-b2)^2]}$$

Where:

- L1, a1, b1 are the reference Lab values.
- L2, a2, b2 are the observed pixel Lab values.
- ΔE76 represents the distance between the two colors in Lab space.

A larger ΔE76 indicates a larger color difference.

3.5.3 Directional color difference

The assignment describes comparison with a healthy steel-alloy calibration profile. This can be implemented by calculating the difference between the observed Lab values and the reference values.

For example:

$$\Delta L = L_{observed} - L_{reference}$$

$$\Delta a = a_{observed} - a_{reference}$$

$$\Delta b = b_{observed} - b_{reference}$$

The direction of the change is important.

A suspected rust region may show a positive change along the a* axis or b* axis, depending on the material, illumination and actual corrosion color.

The system should not classify a region as rust only because its ΔE value is large. A shadow, stain, oil mark or reflection may also produce a large color difference.

Therefore, the system should combine color difference with additional features such as:

- Region size.
- Local texture.
- Shape.
- Boundary characteristics.
- Position on the battery casing.
- Comparison with nearby healthy material.

3.5.4 Advantages of Lab

1. It separates lightness from two chromatic color axes.
2. It supports direct calculation of color difference.
3. It is useful for comparing a surface with a calibrated reference.
4. It can help identify subtle color changes.
5. It provides a structured way to compare different regions.
6. It is suitable for combining color information with texture and shape features.

3.5.5 Limitations of Lab

Lab does not automatically remove the effects of illumination.

The conversion from camera RGB values to Lab depends on the assumed or configured color reference and white point. Different lighting conditions can change the measured Lab values.

Therefore, the system requires controlled lighting, stable camera settings, white-balance calibration and a reference target.

Other limitations include:

- Additional computation for color conversion.
- Incorrect calibration can create misleading color differences.
- Reflections may still affect measured values.
- A simple $\Delta E76$ threshold may not distinguish rust from oil or other stains.
- The color of rust can vary with battery material and corrosion condition.

3.5.6 Suitability for GreenCycle Robotics

Lab is a strong candidate for the main color-comparison stage because the system must identify subtle color differences from a healthy casing reference.

It should be combined with controlled illumination and local texture analysis. A calibrated Lab-based comparison is more appropriate than relying only on raw BGR thresholding.

For applications requiring more perceptually refined color difference, CIEDE2000 may also be evaluated. However, the simpler ΔE76 method is suitable for explaining and implementing the basic reference-comparison approach in this assignment.

3.6 Comparison of RGB/BGR, HSV and Lab

Parameter	**Raw RGB/BGR**	**HSV**	**CIE L*ab****
Components	Red, Green, Blue	Hue, Saturation, Value	Lightness, a*, b*
Brightness separation	No	Yes, through Value	Yes, through L*
Color thresholding	Possible	Convenient	Possible
Color difference calculation	Direct channel distance	Possible but less perceptually uniform	ΔE calculation
Illumination sensitivity	High	Moderate; depends on saturation and exposure	Still present; calibration required
Reference comparison	Possible	Possible	Well suited
Processing complexity	Low	Low to moderate	Moderate
Main limitation	Channel values change with lighting	Hue instability at low saturation	Requires correct calibration
Application suitability	Initial input	Color segmentation	Calibrated rust comparison

Overall observation: BGR is useful as the original image format, HSV is useful for color segmentation, and Lab is useful for measuring color differences against a calibrated healthy-surface reference.

3.7 Effect of Illumination Changes

3.7.1 Importance of illumination stability

The GreenCycle facility has a wide ambient illumination range:

- Minimum ambient illumination: 1,500 lux.

- Maximum ambient illumination: 45,000 lux.

This is a large change in the amount of light reaching the inspection surface.

If the camera uses fixed exposure settings, the pixel values may change with illumination. If automatic exposure is enabled, the camera may adjust its exposure time or gain, which can also affect image consistency and motion blur.

Therefore, the inspection system should use controlled lighting wherever possible.

3.7.2 Baseline illumination – 1.0×

At baseline illumination, the camera captures the battery casing under the reference lighting condition.

This image is used to establish:

- Healthy casing reference values.
- Expected rust color distribution.
- Normal surface texture.
- Reference brightness.
- Color-difference thresholds.

The reference profile should be created using representative samples rather than a single pixel.

3.7.3 Dim illumination – 0.4×

At 0.4× baseline intensity, the available illumination is reduced to 40% of the baseline level.

If exposure and gain remain fixed, image intensity will generally decrease. This can make dark rust spots harder to distinguish from the surrounding casing.

Expected effects include:

- Lower image brightness.
- Increased influence of sensor noise.
- Reduced visibility of subtle texture.
- Possible instability in Hue when saturation becomes low.
- Changes in measured Lab values.

A fixed threshold created under baseline lighting may therefore fail under dim illumination.

3.7.4 Bright illumination – 1.8×

At 1.8× baseline intensity, the illumination is increased to 180% of the baseline level.

Possible effects include:

- Increased pixel intensity.
- Loss of detail in bright regions if values become clipped.
- Stronger reflections from metallic surfaces.
- Changes in measured saturation.
- Different Lab values compared with the reference condition.

The system should avoid overexposure because clipped pixels cannot preserve the original surface detail.

3.7.5 Expected comparison under the three illumination conditions

Illumination condition	Relative intensity	Expected BGR behaviour	Expected HSV behaviour	Expected Lab behaviour
Baseline	1.0×	Reference channel values	Reference Hue/Saturation/Value	Reference Lab profile
Dim	0.4×	Channel intensities generally decrease	Value decreases; Hue may become unstable at low saturation	L* and possibly a*/b* values may shift
Bright	1.8×	Channel intensities generally increase until clipping	Value increases; reflections may reduce saturation	L* and chromatic values may shift

These are expected tendencies, not measured experimental results. Actual values must be recorded from the inspection camera using the same battery samples and controlled test conditions.

3.7.6 Recommended experimental procedure

1. Select representative healthy and rusted battery casing samples.
2. Capture all samples under baseline illumination.
3. Capture the same samples under 0.4× illumination.
4. Capture the same samples under 1.8× illumination.
5. Keep camera position, focus and image resolution constant.
6. Record exposure time, gain and white-balance settings.
7. Convert every image into BGR, HSV and Lab representations.
8. Select the same physical regions in each image.

9. Calculate the average color values for each region.
10. Calculate the difference between healthy and rusted regions.
11. Compare the separation between the two groups.
12. Record false detections and missed rust regions.

The experiment should be repeated using several samples to avoid drawing conclusions from one battery.

3.8 Cluster Dispersion and Color Separation

Color clustering is a technique used to group pixels with similar color characteristics.

For this assignment, two important groups are:

- Healthy casing pixels.
- Rust-affected pixels.

A good color representation should make these two groups easier to separate.

3.8.1 Cluster centre

The cluster centre represents the average color of a group of pixels.

For a group containing N pixels, the mean value of one color component can be calculated as:

$$\text{Mean} = (x1 + x2 + ... + xN) / N$$

The same calculation can be applied independently to the L*, a* and b* components.

3.8.2 Cluster dispersion

Cluster dispersion describes how widely the pixel values are spread around their average.

A simple variance calculation is:

$$\text{Variance} = \Sigma(xi - \text{mean})^2 / N$$

A low variance indicates that the pixel values are relatively close to their mean. A high variance indicates a wider spread.

3.8.3 Comparing healthy and rust clusters

The system should compare:

1. The distance between the healthy and rust cluster centres.
2. The spread of each cluster.
3. The amount of overlap between the clusters.

If the two groups have a large separation and relatively small dispersion, color-based classification may be easier.

If the groups overlap significantly, color alone may not be sufficient. Texture, shape and spatial information should be included.

3.8.4 Suggested cluster analysis diagram

Figure 3.1 – Conceptual cluster separation in Lab color space

- X-axis: a* component.
- Y-axis: b* component.
- Healthy casing pixels: one cluster.
- Rust-affected pixels: another cluster.
- Cluster centres: marked using separate symbols.
- Distance between centres: indicates color separation.

The diagram should be based on actual sample measurements if experimental data is available. Otherwise, label it clearly as a conceptual illustration.

3.9 NumPy Image Array Representation

Python uses NumPy arrays to store and process image data.

OpenCV images are commonly represented as NumPy arrays with the following properties:

- Data type: uint8.
- Dimensions: height × width × 3.
- Channel order: BGR.
- Value range: 0 to 255.

Example:

```
import cv2

import numpy as np

image = cv2.imread("battery.jpg")

print(image.shape)

print(image.dtype)
```

If the image is successfully loaded, the output may look like:

```
(480, 640, 3)
```

uint8

This is an illustrative example. The actual dimensions depend on the image file.

3.9.1 Memory calculation

For an image of height H, width W and three 8-bit channels:

Memory = H × W × 3 × 1 byte

For a 640 × 480 image:

Memory = 640 × 480 × 3

Memory = 921,600 bytes

This is the approximate uncompressed pixel-array memory requirement.

The actual memory usage may be different when using other data types or additional temporary arrays during processing.

3.10 NumPy Addition and OpenCV Saturation

An important issue in image processing is the difference between NumPy arithmetic and OpenCV image arithmetic.

Both can add pixel values, but their behaviour can be different when values exceed the maximum value of an 8-bit image.

3.10.1 NumPy uint8 addition

A uint8 value can store integers from 0 to 255.

When addition exceeds 255, unsigned 8-bit arithmetic can wrap around.

For example:

250 + 10 = 260

But 260 cannot be represented in an 8-bit unsigned integer. The result wraps around modulo 256:

260 - 256 = 4

Therefore, the resulting value may become 4 instead of 255.

This can produce incorrect image results if it is not handled properly.

3.10.2 OpenCV saturated addition

OpenCV's cv2.add() uses saturated arithmetic for standard 8-bit image inputs.

For example:

cv2.add(250, 10) = 255

The result is limited to the maximum allowed value instead of wrapping around to 4.

This is important when combining images, adding brightness or applying image corrections.

3.10.3 Comparison

Input values NumPy uint8 addition OpenCV cv2.add

100 + 20	120	120
200 + 30	230	230
250 + 10	4 after wraparound	255
255 + 20	19 after wraparound	255

3.10.4 Safe arithmetic using a larger data type

A safer approach is to convert the image into a larger data type before performing arithmetic.

Example:

image16 = image.astype(np.uint16)

After processing, the result can be limited to the valid 8-bit range:

result = np.clip(image16, 0, 255).astype(np.uint8)

This prevents unexpected wraparound during intermediate calculations.

3.11 Python Implementation Using OpenCV

The following program reads a battery image, converts it into HSV and Lab color spaces, and calculates a basic Lab color-difference map using a healthy reference color.

Program 3.1 – Color-space conversion and Lab color difference

```
import cv2

import numpy as np


# Read the battery image

image = cv2.imread("battery.jpg")


if image is None:

    raise FileNotFoundError("Could not load battery.jpg")
```

```
# Convert BGR image into HSV
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)

# Convert BGR image into Lab
lab = cv2.cvtColor(image, cv2.COLOR_BGR2LAB)

# Display image information
print("Image shape:", image.shape)
print("Image data type:", image.dtype)

# Example reference Lab value.
# Replace this with the mean Lab value measured
# from a healthy casing calibration region.
reference = np.array([120, 130, 128], dtype=np.float32)

# Convert Lab image to float for calculations
lab_float = lab.astype(np.float32)

# Calculate Lab difference for every pixel
difference = lab_float - reference

# Calculate Euclidean Delta E (CIE76)
delta_e = np.sqrt(np.sum(difference ** 2, axis=2))

# Normalize the result for display
delta_display = cv2.normalize(
    delta_e,
    None,
```

```
    0,
    255,
    cv2.NORM_MINMAX
).astype(np.uint8)

# Create a sample threshold mask
threshold = 15
rust_candidate_mask = (delta_e > threshold).astype(np.uint8) * 255

# Display results
cv2.imshow("Original BGR Image", image)
cv2.imshow("HSV Image", hsv)
cv2.imshow("Lab Image", lab)
cv2.imshow("Lab Delta E Map", delta_display)
cv2.imshow("Candidate Difference Mask", rust_candidate_mask)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Important implementation note: The reference values and threshold in this example are illustrative. The reference must be measured from healthy casing samples captured using the actual camera and calibrated lighting. A large ΔE value indicates a color difference, not necessarily rust. Additional filtering and classification are required to distinguish corrosion from oil, stains, reflections and shadows.

3.12 Improved Rust Detection Workflow

A more reliable rust-detection workflow can combine Lab color comparison with other image-processing operations.

Step 1 – Image acquisition

Capture the battery image using a fixed camera position and controlled illumination.

Step 2 – Image calibration

Use a suitable color reference target and stable camera settings to establish a repeatable color profile.

Step 3 – Color-space conversion

Convert the original BGR image into Lab.

Step 4 – Reference comparison

Compare each relevant casing region with the healthy reference Lab profile.

Step 5 – Candidate region extraction

Generate a candidate mask based on the color-difference threshold.

Step 6 – Noise removal

Use morphological operations to remove isolated pixels and small noise regions.

Step 7 – Region analysis

Extract contours and calculate area, shape and boundary characteristics.

Step 8 – Final decision

Combine color difference with texture and shape information to decide whether a region is likely to be rust.

Step 9 – Evidence storage

Store the original image, detected region, decision result and timestamp for later inspection.

Figure 3.2 – Proposed Lab-based rust detection pipeline

Battery Image Acquisition

|

v

Camera and Color Calibration

|

v

Original BGR Image

|

v

BGR to CIE L*a*b* Conversion

|

v

Healthy Casing Reference Comparison

|

v

ΔE Map

|

v

Candidate Rust Segmentation

|

v

Noise Removal and Morphology

|

v

Contour and Texture Analysis

|

v

Rust / Non-Rust Decision

|

v

Inspection Record and Evidence

3.13 Experimental Evaluation Plan

The assignment requires comparison under three illumination conditions. A proper evaluation should use the same samples under all conditions.

3.13.1 Test dataset

The test dataset should contain:

- Healthy battery casings.
- Lightly rusted casings.
- Moderately rusted casings.

- Severely rusted casings.
- Oil-stained casings.
- Scratched casings.
- Reflective casings.
- Casings with shadows.

The dataset should include cylindrical 18650 cells, cylindrical 21700 cells and prismatic pouch cells where the casing material makes rust inspection applicable.

3.13.2 Evaluation measurements

The following measurements can be recorded:

Measurement	Purpose
Mean healthy-region Lab value	Establish reference profile
Mean rust-region Lab value	Measure rust color
Mean ΔE76	Measure color difference
Cluster variance	Measure color spread
Cluster centre distance	Measure group separation
False-positive rate	Measure healthy regions incorrectly marked as rust
False-negative rate	Measure rust regions missed by the system
Processing time	Measure computational cost

3.13.3 Results table template

The following table should be completed after capturing and processing the test images.

Illumination	Color space	Mean healthy value	Mean rust value	Cluster separation	False detections
1.0×	BGR	To be measured	To be measured	To be measured	To be measured
1.0×	HSV	To be measured	To be measured	To be measured	To be measured
1.0×	Lab	To be measured	To be	To be measured	To be

			measured		measured
0.4×	BGR	To be measured	To be measured	To be measured	To be measured
0.4×	HSV	To be measured	To be measured	To be measured	To be measured
0.4×	Lab	To be measured	To be measured	To be measured	To be measured
1.8×	BGR	To be measured	To be measured	To be measured	To be measured
1.8×	HSV	To be measured	To be measured	To be measured	To be measured
1.8×	Lab	To be measured	To be measured	To be measured	To be measured

The values must come from actual test images. It would be incorrect to report invented accuracy percentages or cluster-separation values as experimental results.

3.14 Additional Engineering Considerations

3.14.1 Controlled illumination

The system should use an enclosed inspection area or suitable light shielding to reduce the influence of external light.

3.14.2 Diffuse lighting

A diffuse light source can reduce harsh reflections from metallic battery surfaces. This makes surface color measurements more consistent.

3.14.3 Fixed camera settings

Exposure, gain, focus and white balance should be configured consistently during production inspection. Automatic adjustments should be carefully evaluated because they may change the appearance of the same defect.

3.14.4 Flat-field correction

If the captured image has uneven brightness, flat-field correction can be used to reduce gradual illumination variations.

3.14.5 Combination with texture analysis

Rust may have a different surface texture from healthy metal. Combining color difference with texture measurements can improve discrimination between corrosion and other stains.

3.14.6 Threshold validation

Thresholds should be validated using separate test images. A threshold that works on the training samples may not work on new battery models or different corrosion conditions.

3.15 Final Comparison and Engineering Decision

The three approaches have different strengths.

Raw BGR provides the original camera information and is easy to process, but it is highly affected by changes in illumination. HSV separates Hue and Saturation from Value and can help identify reddish-brown regions. However, Hue becomes less reliable when saturation is low. Lab provides lightness and two chromatic axes and supports direct color-difference measurement using ΔE.

For the GreenCycle Robotics system, the recommended approach is to retain BGR as the original acquisition format, use HSV as an optional segmentation aid, and use calibrated Lab color difference as the main color-comparison method for rust candidates.

This selection must be combined with controlled lighting, color calibration, texture analysis and threshold validation. Lab should not be described as completely illumination-invariant. The final system must be tested under baseline, dim and bright conditions before being approved for production use.

3.16 Conclusion

Digital image representation plays an important role in the reliability of a computer vision inspection system. The same physical battery casing can produce different pixel values under different lighting conditions. Therefore, color-space selection must consider both the type of defect and the operating environment.

Raw BGR is simple and useful for image acquisition, but it is sensitive to illumination changes. HSV provides a convenient representation for color segmentation because it separates Hue, Saturation and Value. CIE L*a*b* is suitable for calibrated color comparison because it separates lightness from chromatic components and supports ΔE measurement.

For the GreenCycle battery-sorting application, a calibrated Lab-based approach is recommended for detecting subtle oxide rust. It should be supported by controlled lighting, reference calibration, image correction and additional texture analysis. Actual performance must be established using a representative test dataset under all three specified illumination conditions.

3.17 Suggested Figures and Diagrams

Include the following figures in this task to improve its technical presentation:

Figure 3.1: RGB/BGR color model showing the three color channels.

Figure 3.2: HSV color model showing Hue, Saturation and Value.

Figure 3.3: CIE L*a*b* color-space diagram showing the L*, a* and b* axes.

Figure 3.4: Same battery casing photographed under 0.4×, 1.0× and 1.8× illumination.

Figure 3.5: Comparison of BGR, HSV and Lab representations of a rusted battery surface.

Figure 3.6: Conceptual healthy-casing and rust-pixel clusters in Lab space.

Figure 3.7: Lab ΔE color-difference heat map.

Figure 3.8: Complete Lab-based rust detection pipeline.

Figure 3.9: NumPy image-array structure showing rows, columns and channels.

Figure 3.10: Comparison of NumPy wraparound addition and OpenCV saturated addition.

These figures should be numbered consistently and placed near the paragraphs where they are discussed.

TASK 4 – TOPOLOGICAL ANALYSIS: PIXEL CONNECTIVITY FOR CASING SCRATCH COUNTING

4.1 Introduction

In computer vision, an image is made up of pixels arranged in rows and columns. When a defect such as a scratch appears on a battery casing, it is represented by a group of connected pixels. Identifying how these pixels are connected is important for detecting, separating and counting individual defects.

In the GreenCycle Robotics battery inspection system, scratches may be very thin and may contain diagonal or curved structures. If the computer vision system uses an unsuitable pixel-connectivity method, one scratch may be divided into multiple regions or two separate scratches may incorrectly be treated as one defect.

This task compares three pixel-connectivity methods:

1. 4-connectivity (N4).
2. 8-connectivity (N8).
3. Mixed connectivity (m-connectivity).

It also compares three distance measurements:

- Euclidean distance.
- City-Block distance.
- Chessboard distance.

The objective is to identify a suitable connectivity and distance-measurement approach for counting scratches and measuring their physical dimensions.

4.2 Understanding Pixel Connectivity

Pixel connectivity determines whether two pixels are considered neighbours.

Consider a binary image containing two types of values:

- 0 represents the background.
- 1 represents a foreground object, such as a scratch.

A foreground pixel may be connected to another foreground pixel depending on its position.

For example, two pixels next to each other horizontally are connected under 4-connectivity. Two pixels touching only at a corner are not connected under 4-connectivity, but they are connected under 8-connectivity.

Connectivity is important because connected-component analysis uses these neighbourhood relationships to identify separate objects.

4.3 Technique A – 4-Connectivity (N4)

4.3.1 Working principle

In 4-connectivity, a pixel is connected only to its four direct neighbours:

- Top.
- Bottom.
- Left.
- Right.

For a pixel at position (x, y), its four neighbours are:

N4(p) = {(x-1,y), (x+1,y), (x,y-1), (x,y+1)}

Diagonal pixels are not included.

4.3.2 Example

Consider the following binary image:

1 0

0 1

The two foreground pixels touch diagonally. Under 4-connectivity, they are treated as two separate components because neither pixel is a horizontal or vertical neighbour of the other.

4.3.3 Advantages of 4-connectivity

1. It provides a simple neighbourhood structure.
2. It avoids joining objects that touch only at diagonal corners.
3. It can reduce accidental merging of nearby defects.
4. It is computationally simple.
5. It is useful when defects are expected to have horizontal or vertical pixel connections.

4.3.4 Limitations of 4-connectivity

1. Thin diagonal scratches may be divided into multiple components.
2. A continuous diagonal line may appear as several separate objects.
3. Scratch counting may become inaccurate.
4. Small gaps between diagonally adjacent pixels can affect defect measurements.

4.3.5 Suitability for GreenCycle Robotics

4-connectivity may be useful when the system must keep diagonally touching objects separate. However, it is not ideal as the only connectivity rule for scratch counting because scratches can run diagonally across the battery casing.

4.4 Technique B – 8-Connectivity (N8)

4.4.1 Working principle

In 8-connectivity, a pixel is connected to all eight surrounding neighbours.

These include:

- Top.
- Bottom.
- Left.
- Right.
- Top-left.
- Top-right.
- Bottom-left.
- Bottom-right.

The neighbourhood is represented as:

N8(p) = N4(p) + diagonal neighbours

4.4.2 Example

Consider the same binary image:

1 0

0 1

Under 8-connectivity, the two foreground pixels are treated as one connected component because they touch diagonally.

4.4.3 Advantages of 8-connectivity

1. It preserves diagonal lines more effectively.
2. It is suitable for detecting thin diagonal scratches.
3. It reduces fragmentation of continuous diagonal defects.
4. It supports connected-component labelling.
5. It is useful for irregularly shaped defects.

4.4.4 Limitations of 8-connectivity

1. Nearby defects touching at a corner may be merged.
2. Diagonal noise pixels may join otherwise separate regions.
3. Two separate scratches may be counted as one component.
4. It may require additional filtering to remove accidental connections.

4.4.5 Suitability for GreenCycle Robotics

8-connectivity is useful for detecting continuous scratches that run in different directions. However, the system must examine the shape and distance between components to avoid merging separate defects.

4.5 Technique C – Mixed Connectivity (m-Connectivity)

4.5.1 Working principle

Mixed connectivity is designed to reduce ambiguity when diagonal pixels are considered connected.

It uses both direct and diagonal neighbourhood relationships. A diagonal connection is accepted only under a specific condition involving the common 4-neighbours of the two pixels.

Two pixels p and q are m-adjacent when:

1. They are 4-neighbours, or
2. They are diagonal neighbours and their common 4-neighbours do not contain a foreground pixel.

This rule helps avoid some unwanted connections created by ordinary 8-connectivity.

4.5.2 Example

Consider a diagonal pair of foreground pixels. If their common 4-neighbours are background pixels, the diagonal connection can be accepted.

If a common 4-neighbour is also a foreground pixel, the diagonal connection is not accepted under the mixed-connectivity rule.

This helps create a more controlled connection structure.

4.5.3 Advantages of mixed connectivity

1. It reduces certain connectivity ambiguities.
2. It can prevent some unwanted diagonal merging.
3. It combines direct and conditional diagonal connections.
4. It is useful for analysing complex binary patterns.
5. It can provide more controlled connected-component relationships.

4.5.4 Limitations of mixed connectivity

1. It is more complicated to implement than basic 4- or 8-connectivity.
2. It requires checking additional neighbourhood conditions.
3. It may still require noise removal and morphological processing.
4. It is not automatically the best choice for every scratch pattern.

4.5.5 Suitability for GreenCycle Robotics

Mixed connectivity is a useful option when diagonal connections create ambiguity. It can help prevent unwanted joining of defects, but its performance should be compared with 4- and 8-connectivity using representative scratch images.

4.6 Comparison of 4-, 8- and Mixed Connectivity

Parameter	**4-Connectivity**	**8-Connectivity**	**Mixed Connectivity**
Direct neighbours	4	4	4
Diagonal neighbours	Not included	Included	Included conditionally
Diagonal scratch detection	May fragment	Usually preserves connections	Depends on m-adjacency condition
Accidental diagonal	Lower	Higher possibility	Reduced in some

merging			patterns
Implementation	Simple	Simple	More complex
Main advantage	Separates corner-touching pixels	Preserves diagonal structures	Reduces connectivity ambiguity
Main limitation	Diagonal fragmentation	Unwanted merging	More complicated processing

4.7 Synthetic Scratch Patterns for Testing

Before applying connectivity rules to actual battery images, synthetic binary images can be created. These images help test how each method behaves under controlled conditions.

Three important test patterns are:

4.7.1 Diagonal one-pixel lines

A diagonal line can be represented as:

1 0 0 0 0

0 1 0 0 0

0 0 1 0 0

0 0 0 1 0

0 0 0 0 1

Under 4-connectivity, the pixels are separate components because they touch only diagonally.

Under 8-connectivity, they form one connected component.

Mixed connectivity evaluates the diagonal relationships using its additional condition.

This test is important because real scratches may be thin and diagonal.

4.7.2 Checkerboard patterns

A checkerboard pattern contains alternating foreground and background pixels.

1 0 1 0

0 1 0 1

1 0 1 0

0 1 0 1

Under 4-connectivity, the foreground pixels are isolated from one another.

Under 8-connectivity, many foreground pixels become connected through diagonal relationships.

This pattern demonstrates why 8-connectivity can create unwanted connections in dense pixel arrangements.

4.7.3 Noisy backgrounds

A noisy binary image may contain isolated foreground pixels around a scratch.

These pixels can create false connected components or accidentally join nearby defects.

Noise removal can be performed using morphological operations and component-area filtering.

The system should compare component counts before and after filtering to understand the effect of noise removal.

4.8 Connected-Component Analysis

Connected-component analysis is used to identify separate groups of foreground pixels in a binary image.

Each connected region receives a label. The background normally receives label 0, while detected foreground regions receive other labels.

For scratch counting, the system can use the following procedure:

1. Capture the battery casing image.
2. Convert the image into a suitable grayscale or color representation.
3. Apply preprocessing to reduce noise.
4. Generate a binary image using a suitable segmentation method.
5. Apply connected-component labelling.
6. Calculate the area of each component.
7. Remove very small noise components.
8. Count the remaining scratch candidates.
9. Measure their dimensions using calibrated image coordinates.

OpenCV provides connected-component functions that support 4- and 8-connectivity.

4.9 Distance Measurement in Digital Images

Connectivity determines whether pixels are connected. Distance metrics determine how far two pixels are from each other.

Distance measurement is important for estimating scratch length, width and separation.

Three common distance metrics are:

1. Euclidean distance.
2. City-Block distance.
3. Chessboard distance.

4.10 Euclidean Distance (De)

4.10.1 Working principle

Euclidean distance is the straight-line distance between two points.

For two pixels:

P1 = (x1, y1)

P2 = (x2, y2)

The distance is:

$De = \sqrt{[(x2-x1)^2 + (y2-y1)^2]}$

This measurement represents the direct geometric distance between the two points.

4.10.2 Example

Consider two points:

P1 = (2, 3)

P2 = (5, 7)

The Euclidean distance is:

$De = \sqrt{[(5-2)^2 + (7-3)^2]}$

$De = \sqrt{(9 + 16)}$

$De = \sqrt{25}$

De = 5 pixels

4.10.3 Advantages

1. It measures straight-line distance.
2. It corresponds to ordinary geometric distance.
3. It is suitable for measuring physical dimensions after camera calibration.
4. It does not favour horizontal, vertical or diagonal directions.
5. It is useful for estimating the distance between scratch endpoints.

4.10.4 Limitations

1. It measures the straight-line distance between two points, not necessarily the length of a curved scratch.
2. It requires coordinate calibration to convert pixels into millimetres.
3. Perspective distortion can affect physical measurements.
4. Irregular scratches may require skeleton-based path-length measurements instead.

4.11 City-Block Distance (D4)

4.11.1 Working principle

City-Block distance is also called Manhattan distance.

It measures the distance between two points by adding the horizontal and vertical movements required to reach one point from another.

The formula is:

$D4 = |x2-x1| + |y2-y1|$

4.11.2 Example

Using:

$P1 = (2, 3)$

$P2 = (5, 7)$

The City-Block distance is:

$D4 = |5-2| + |7-3|$

$D4 = 3 + 4$

$D4 = 7$ pixels

4.11.3 Advantages

1. It is simple to calculate.
2. It follows horizontal and vertical pixel movements.
3. It is related to 4-connectivity.
4. It can be useful for grid-based processing.

4.11.4 Limitations

1. It overestimates diagonal straight-line distance.
2. It produces direction-dependent measurements.
3. It does not represent the shortest physical distance between two points.

4. It is less suitable for measuring diagonal scratch dimensions.

4.12 Chessboard Distance (D8)

4.12.1 Working principle

Chessboard distance is based on the number of horizontal, vertical or diagonal moves needed to reach one point from another.

The formula is:

$D8 = \max(|x2-x1|, |y2-y1|)$

4.12.2 Example

Using:

P1 = (2, 3)

P2 = (5, 7)

The Chessboard distance is:

D8 = max(3, 4)

D8 = 4 pixels

4.12.3 Advantages

1. It supports diagonal movement.
2. It is related to 8-connectivity.
3. It is simple to calculate.
4. It can be useful for neighbourhood-based image processing.

4.12.4 Limitations

1. It does not measure ordinary straight-line distance.
2. It can underestimate diagonal physical distance.
3. It produces direction-dependent geometric measurements.
4. It is not the preferred metric for reporting calibrated physical scratch length.

4.13 Comparison of Distance Metrics

Parameter	**Euclidean (De)**	**City-Block (D4)**	**Chessboard (D8)**
Formula	$\sqrt{(dx^2 + dy^2)}$	$\|dx\| + \|dy\|$	$\max(\|dx\|,\|dy\|)$

Movement	Straight line	Horizontal and vertical	Horizontal, vertical and diagonal
Diagonal handling	Direct geometric distance	Counts both axis movements	Counts diagonal steps as one
Direction dependence	No, in the image plane	Yes	Yes
Suitable for physical endpoint distance	Yes, after calibration	Less suitable	Less suitable
Main application	Geometric measurement	Grid-based processing	Neighbourhood processing

For the example with coordinate differences of 3 pixels horizontally and 4 pixels vertically:

- Euclidean distance = 5 pixels.
- City-Block distance = 7 pixels.
- Chessboard distance = 4 pixels.

This demonstrates that the three metrics produce different results for the same pair of points.

4.14 Physical Dimension Measurement Using Camera Calibration

Pixel measurements must be converted into physical measurements before the system can report scratch length in millimetres.

The GreenCycle camera has a field of view of 320 mm across the conveyor and 240 mm along the conveyor.

If the image width is W pixels, the approximate horizontal scale is:

Horizontal scale = 320 / W mm per pixel

If the image height is H pixels, the approximate vertical scale is:

Vertical scale = 240 / H mm per pixel

For example, if a captured image is 1920 × 1080 pixels:

Horizontal scale = 320 / 1920

= 0.1667 mm/pixel approximately

Vertical scale = 240 / 1080

= 0.2222 mm/pixel approximately

These are illustrative calculations based on the stated field of view and an assumed 1920 × 1080 image.

The horizontal and vertical scales are not equal in this example. Therefore, the image should not be treated as having square physical sampling units.

4.14.1 Corrected Euclidean distance

When horizontal and vertical scales are different, the physical distance should be calculated using the calibrated scale values.

$$D_{physical} = \sqrt{[(dx \times sx)^2 + (dy \times sy)^2]}$$

Where:

- dx is the horizontal pixel difference.
- dy is the vertical pixel difference.
- sx is the horizontal scale in mm/pixel.
- sy is the vertical scale in mm/pixel.

This method avoids treating unequal horizontal and vertical sampling distances as identical.

4.14.2 Example

Assume:

- Horizontal scale = 0.1667 mm/pixel.
- Vertical scale = 0.2222 mm/pixel.
- Horizontal pixel difference = 3.
- Vertical pixel difference = 4.

Then:

$$D_{physical} = \sqrt{[(3 \times 0.1667)^2 + (4 \times 0.2222)^2]}$$

$$D_{physical} = \sqrt{[(0.5001)^2 + (0.8888)^2]}$$

$$D_{physical} \approx 1.02 \text{ mm}$$

This is the approximate physical straight-line distance between the two measured points.

The result assumes that the image has been geometrically calibrated and that perspective effects are negligible or corrected.

4.15 Measuring Curved Scratch Length

A scratch may not be a straight line. Therefore, measuring only the distance between its two endpoints may underestimate its actual length.

For curved scratches, the system can use skeletonization.

Skeletonization reduces a binary scratch region to a thin centreline while preserving its general shape.

The centreline can then be measured by adding the distances between successive connected pixels.

For an 8-connected skeleton:

- Horizontal or vertical steps have a length of one pixel unit.
- Diagonal steps have a length of $\sqrt{2}$ pixel units when pixels have equal physical spacing.

If horizontal and vertical scales are different, the physical length of each step should be calculated using the calibrated horizontal and vertical scale values.

For example:

Horizontal step length:

sx

Vertical step length:

sy

Diagonal step length:

$\sqrt{(sx^2 + sy^2)}$

The total scratch length is the sum of the physical lengths of the successive skeleton steps.

This method is more suitable for measuring curved scratch paths than using only the endpoint distance.

4.16 Python Implementation Using OpenCV

The following program creates a synthetic diagonal scratch pattern and compares connected-component counts using 4- and 8-connectivity.

Program 4.1 – Comparing 4-connectivity and 8-connectivity

```
import cv2

import numpy as np


# Create a black binary image

image = np.zeros((200, 200), dtype=np.uint8)
```

```
# Draw a one-pixel diagonal line
for i in range(30, 150, 10):
    image[i, i] = 255

# Connected components using 4-connectivity
count4, labels4, stats4, centroids4 = \
    cv2.connectedComponentsWithStats(
        image,
        connectivity=4
    )

# Connected components using 8-connectivity
count8, labels8, stats8, centroids8 = \
    cv2.connectedComponentsWithStats(
        image,
        connectivity=8
    )

print("Components with 4-connectivity:", count4 - 1)
print("Components with 8-connectivity:", count8 - 1)

cv2.imshow("Original Diagonal Pattern", image)
cv2.imshow("4-Connectivity Labels",
           (labels4 * 30).astype(np.uint8))
 cv2.imshow("8-Connectivity Labels",
           (labels8 * 30).astype(np.uint8))

cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

The background component is included in the count returned by OpenCV, so one is subtracted to obtain the foreground component count.

For this diagonal pattern, 4-connectivity should identify separate foreground components, while 8-connectivity should join the diagonal pixels into one component.

4.17 Python Implementation for Distance Metrics

Program 4.2 – Euclidean, City-Block and Chessboard distance

```
import math


# Define two pixel coordinates

x1, y1 = 2, 3

x2, y2 = 5, 7


dx = abs(x2 - x1)

dy = abs(y2 - y1)


# Euclidean distance

euclidean = math.sqrt(dx**2 + dy**2)


# City-Block distance

city_block = dx + dy


# Chessboard distance

chessboard = max(dx, dy)


print("Euclidean Distance:", euclidean)

print("City-Block Distance:", city_block)

print("Chessboard Distance:", chessboard)
```

Expected output:

Euclidean Distance: 5.0

City-Block Distance: 7

Chessboard Distance: 4

4.18 Scratch Detection and Counting Workflow

The complete scratch-analysis process can be organised into the following stages:

Stage 1 – Image acquisition: Capture a clear image of the battery casing.

Stage 2 – Preprocessing: Reduce image noise and correct illumination where required.

Stage 3 – Scratch segmentation: Separate possible scratch pixels from the background.

Stage 4 – Binary image creation: Convert the candidate scratch regions into a binary mask.

Stage 5 – Connectivity selection: Apply 4-, 8- or mixed-connectivity rules.

Stage 6 – Connected-component analysis: Identify individual scratch candidates.

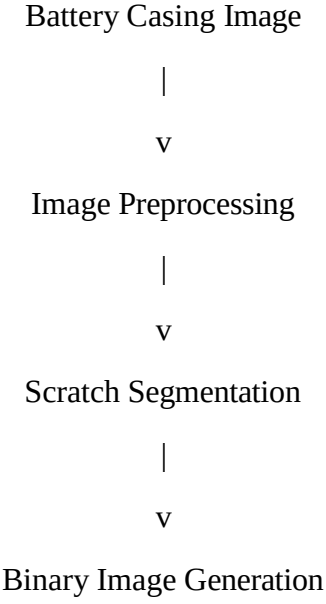
Stage 7 – Noise removal: Remove small components that are unlikely to represent actual scratches.

Stage 8 – Dimension measurement: Calculate length and width using calibrated image coordinates.

Stage 9 – Defect classification: Classify the detected region based on its dimensions and appearance.

Stage 10 – Result storage: Save the detected scratch image, measurements and inspection result.

Figure 4.1 – Scratch detection and counting pipeline



|

v

Connectivity Analysis

|

v

Connected Components

|

v

Noise and Small Region Removal

|

v

Scratch Dimension Measurement

|

v

Defect Count and Classification

|

v

Inspection Result Storage

4.19 Testing and Evaluation Plan

The three connectivity methods should be evaluated using the same synthetic and real scratch images.

4.19.1 Test cases

Test case	**Purpose**
Horizontal one-pixel line	Test direct pixel connectivity
Vertical one-pixel line	Test vertical connections
Diagonal one-pixel line	Identify diagonal fragmentation
Checkerboard pattern	Test unwanted diagonal joining

Two nearby scratches	Check whether separate defects are merged
Noisy background	Evaluate false connected components
Curved scratch	Test scratch-length measurement
Multiple crossing scratches	Evaluate component separation

4.19.2 Evaluation measurements

The following values should be recorded:

- Number of connected components.
- Number of actual scratches.
- False component count.
- Merged scratch count.
- Fragmented scratch count.
- Measured scratch length.
- Difference between measured and reference physical length.
- Processing time.

4.19.3 Result table template

Test pattern	**4-connectivity result**	**8-connectivity result**	**Mixed-connectivity result**
Horizontal line	To be tested	To be tested	To be tested
Vertical line	To be tested	To be tested	To be tested
Diagonal line	To be tested	To be tested	To be tested
Checkerboard	To be tested	To be tested	To be tested
Nearby scratches	To be tested	To be tested	To be tested
Noisy background	To be tested	To be tested	To be tested

Actual results should be filled in after running the programs on the selected test patterns.

4.20 Engineering Recommendation

For the GreenCycle Robotics system, 8-connectivity is a useful starting point for identifying thin diagonal scratches because it preserves diagonal pixel connections.

However, 8-connectivity may merge separate defects that touch diagonally. Therefore, connected-component results should be followed by region analysis, distance checks and shape-based filtering.

Mixed connectivity can be evaluated when diagonal merging becomes a major problem. 4-connectivity can be retained as a comparison method, especially for identifying unwanted diagonal connections.

For physical scratch measurement, Euclidean distance is the preferred basic metric because it measures straight-line geometric distance. The calculation must use calibrated horizontal and vertical scales rather than assuming that every pixel represents the same physical distance in both directions.

For curved scratches, skeleton-based path-length measurement is more suitable than measuring only the distance between endpoints.

4.21 Conclusion

Pixel connectivity is an important part of binary image analysis because it determines how neighbouring pixels are grouped into connected regions. In the GreenCycle Robotics battery inspection system, the choice of connectivity directly affects scratch counting and defect measurement.

4-connectivity considers only horizontal and vertical neighbours and may divide diagonal scratches into multiple components. 8-connectivity includes diagonal neighbours and preserves diagonal structures, but it may merge nearby defects. Mixed connectivity provides a conditional approach to diagonal connections and can reduce some unwanted merging.

Among the three distance metrics, Euclidean distance is the most appropriate basic measurement for straight-line physical dimensions after camera calibration. City-Block and Chessboard distances are useful for grid-based image processing but can produce direction-dependent measurements.

The recommended system should begin with 8-connectivity, evaluate mixed connectivity for ambiguous patterns, and use calibrated Euclidean measurement for physical dimensions. Curved scratches should be measured using their skeleton paths. Testing with diagonal lines, checkerboard patterns, nearby scratches and noisy backgrounds is necessary before selecting the final implementation.

4.22 Suggested Figures and Diagrams

Figure 4.2: Diagram showing the four direct neighbours in 4-connectivity.

Figure 4.3: Diagram showing all eight neighbours in 8-connectivity.

Figure 4.4: Diagram explaining the conditional diagonal connection in m-connectivity.

Figure 4.5: Comparison of diagonal scratch detection using 4-, 8- and mixed connectivity.

Figure 4.6: Synthetic checkerboard pattern showing connected-component differences.

Figure 4.7: Euclidean, City-Block and Chessboard distance comparison between two pixels.

Figure 4.8: Camera calibration diagram showing pixel-to-millimetre conversion.

Figure 4.9: Scratch skeletonization and curved path-length measurement.

Figure 4.10: Complete scratch detection and counting workflow.

TASK 5 – PHYSICAL ACQUISITION: SENSOR SHUTTER AND EXPOSURE TOPOLOGY

5.1 Introduction

Physical image acquisition is the process of capturing light reflected from real-world objects and converting it into digital image data. The quality of a computer vision system depends not only on its image-processing algorithms but also on the camera sensor, shutter mechanism, exposure time and lighting arrangement.

In the GreenCycle Robotics battery-sorting system, batteries move along a conveyor at a speed of 1.2 m/s. The camera must capture clear images of moving batteries so that the system can identify their form factors, read chemistry codes and detect physical defects.

The smallest defect that must be detected is 0.3 mm. The laser-etched text has a stroke width of 0.4 mm. Therefore, motion blur must be controlled because excessive blur can hide small defects and reduce text readability.

The system also operates under changing ambient illumination, and overhead 50 Hz grid lighting produces 100 Hz light flicker. These conditions make camera selection and exposure synchronization important.

This task compares three acquisition methods:

1. Area-scan CMOS camera with rolling shutter and constant overhead lighting.
2. Global-shutter CMOS camera with high-intensity LED strobe synchronized with a hardware belt encoder.
3. Line-scan camera triggered by belt displacement.

The objective is to identify a suitable image-acquisition method for high-speed battery inspection.

5.2 Fundamentals of Image Acquisition

Image acquisition begins when light from a source reaches an object and is reflected towards the camera lens.

The lens focuses the incoming light onto the image sensor. The sensor converts the received light into electrical signals. These signals are processed and converted into digital pixel values.

The main components involved are:

1. Lighting source.
2. Battery surface.
3. Camera lens.
4. Image sensor.
5. Shutter mechanism.
6. Exposure control.
7. Analog signal processing.
8. Analog-to-digital converter.
9. Digital image output.

The captured image is then transferred to the edge computer for processing.

5.3 Understanding Shutter Mechanisms

A camera shutter controls how long the image sensor receives light.

The exposure time determines the duration for which the sensor collects light from the scene.

For a stationary object, a longer exposure can collect more light. However, when an object is moving, a longer exposure can cause motion blur.

Two important shutter types used in area-scan cameras are:

- Rolling shutter.
- Global shutter.

A line-scan camera uses a different acquisition arrangement in which image lines are captured sequentially as the object moves.

5.4 Technique A – Rolling-Shutter CMOS Camera with Constant Overhead Lighting

5.4.1 Working principle

A rolling-shutter camera captures an image by exposing and reading different sensor rows at slightly different times.

The sensor does not capture every row at exactly the same instant. Instead, the acquisition process progresses across the image sensor.

This means that the top and bottom portions of a moving battery may represent slightly different moments in time.

When an object moves during the row-readout period, its apparent shape or position may become distorted.

5.4.2 Operation in the GreenCycle system

In Technique A, a CMOS camera is installed above the conveyor and uses constant overhead lighting.

The camera captures battery images while the conveyor moves at 1.2 m/s.

The captured frames are transferred to the edge computer for preprocessing and defect detection.

The main advantage is that rolling-shutter cameras are commonly available and may provide a lower-cost acquisition option.

5.4.3 Advantages

1. Lower initial cost compared with some industrial global-shutter systems.
2. CMOS sensors can provide good image quality.
3. Constant lighting avoids the need for a complex strobe trigger system.
4. Suitable for applications with limited object movement.
5. Camera integration can be relatively simple.
6. Can be used for preliminary inspection and development.

5.4.4 Limitations

1. Different rows are captured at different times.
2. Moving objects may appear geometrically distorted.
3. Fast conveyor movement can cause image deformation.
4. Flickering lighting may create brightness differences between image rows.
5. Longer exposure increases motion blur.
6. Accurate measurement of small defects becomes difficult if the image is distorted.

5.4.5 Suitability for GreenCycle Robotics

A rolling-shutter camera may be suitable for low-speed inspection, but the GreenCycle conveyor operates at 1.2 m/s and requires detection of small defects.

Therefore, rolling-shutter distortion and exposure blur must be evaluated carefully before using this method for production inspection.

5.5 Technique B – Global-Shutter CMOS Camera with LED Strobe

5.5.1 Working principle

A global-shutter camera exposes all sensor pixels at approximately the same time.

Unlike a rolling shutter, the entire image represents nearly the same instant of the moving scene.

This reduces geometric distortion caused by row-by-row exposure timing.

A high-intensity LED strobe can be used to illuminate the battery for a very short period. The strobe is synchronized with a hardware belt encoder or an acquisition trigger.

The short illumination period reduces the time during which the moving battery contributes to the captured image.

5.5.2 Operation in the GreenCycle system

In Technique B, the camera is installed above the conveyor and connected to a hardware trigger.

The belt encoder provides information about conveyor movement. When the battery reaches the required imaging position, the camera is triggered and the LED strobe illuminates the battery.

The camera captures the image using a short exposure.

The global shutter reduces row-timing distortion, while the strobe helps freeze the moving object.

5.5.3 Advantages

1. Captures the image with nearly simultaneous exposure across the sensor.
2. Reduces rolling-shutter geometric distortion.
3. Supports short exposure times.
4. LED strobe can reduce motion blur.
5. Hardware triggering improves repeatability.
6. Encoder synchronization supports consistent image acquisition.
7. Suitable for fast-moving industrial inspection applications.

5.5.4 Limitations

1. Industrial global-shutter cameras may cost more than basic rolling-shutter cameras.
2. High-intensity LED lighting requires suitable power and thermal management.
3. Strobe timing must be configured correctly.
4. Trigger synchronization requires additional integration.
5. Incorrect strobe timing may capture the battery at an unsuitable position.

6. Reflective battery surfaces may still create glare.

5.5.5 Suitability for GreenCycle Robotics

This is the preferred acquisition approach for the GreenCycle system because it reduces rolling-shutter distortion and allows short exposure times.

The combination of global shutter, LED strobe and encoder synchronization is appropriate for capturing moving batteries with small defects.

5.6 Technique C – Line-Scan Camera Triggered by Belt Displacement

5.6.1 Working principle

A line-scan camera captures one line of pixels at a time rather than capturing a complete two-dimensional frame in a single exposure.

As the object moves along the conveyor, successive lines are captured and combined to create a two-dimensional image.

The acquisition rate is synchronized with the movement of the conveyor.

A belt encoder can provide displacement information so that each line is captured after a specified movement distance.

5.6.2 Operation in the GreenCycle system

In Technique C, a line-scan camera is mounted above the conveyor.

The camera continuously captures image lines across the width of the belt. The conveyor movement determines the next line position.

The captured lines are combined to form a complete image of the battery.

This method can be useful for continuous inspection of long objects or large conveyor surfaces.

5.6.3 Advantages

1. Suitable for continuous conveyor inspection.
2. Provides high spatial resolution across the scanning direction.
3. Supports encoder-based displacement synchronization.
4. Can avoid some frame-based coverage limitations.
5. Useful for inspecting long or continuous surfaces.
6. Allows acquisition to be matched to conveyor movement.

5.6.4 Limitations

1. Requires accurate conveyor displacement information.

2. Encoder errors can produce stretched or compressed images.
3. Vibration and object movement can affect reconstructed images.
4. Requires careful synchronization between the camera and conveyor.
5. Lighting must remain consistent during line acquisition.
6. Image reconstruction and integration may be more complicated than area-scan acquisition.

5.6.5 Suitability for GreenCycle Robotics

Line-scan imaging is a strong option for continuous industrial inspection, particularly when accurate encoder synchronization is available.

However, it requires suitable line-rate capability and careful calibration. For the current single-lane battery system, a global-shutter area-scan camera may provide a simpler implementation.

5.7 Comparison of Rolling Shutter, Global Shutter and Line Scan

Parameter	**Rolling-Shutter CMOS**	**Global-Shutter CMOS**	**Line-Scan Camera**
Image acquisition	Rows exposed at different times	Pixels exposed nearly simultaneously	One line at a time
Motion distortion	Possible	Greatly reduced	Depends on encoder synchronization
Short exposure support	Sensor-dependent	Sensor-dependent	Sensor and line-rate dependent
Lighting arrangement	Constant lighting possible	LED strobe recommended	Uniform continuous or synchronized lighting
Triggering	Frame trigger possible	Hardware trigger suitable	Encoder-based triggering
Moving-object suitability	Requires careful evaluation	Very suitable	Very suitable with synchronization
Integration complexity	Relatively low	Moderate	Higher
Main limitation	Rolling-shutter distortion	Trigger and strobe integration	Encoder and line-rate requirements

5.8 Motion Blur Calculation

Motion blur is the distance an object moves during the camera exposure period.

The basic formula is:

Motion Blur = Conveyor Velocity × Exposure Time

Where:

- Motion blur is measured in metres or millimetres.
- Conveyor velocity is measured in metres per second.
- Exposure time is measured in seconds.

For the GreenCycle system:

Conveyor velocity = 1.2 m/s

The assignment specifies three exposure times:

- 4.0 ms.
- 1.0 ms.
- 0.05 ms.

5.8.1 Motion blur at 4.0 ms

Convert milliseconds into seconds:

4.0 ms = 0.004 seconds

Apply the formula:

Blur = 1.2 × 0.004

Blur = 0.0048 m

Blur = 4.8 mm

Therefore, the battery moves 4.8 mm during the exposure.

This amount of movement is too large for reliable inspection of small 0.3 mm defects.

5.8.2 Motion blur at 1.0 ms

Convert milliseconds into seconds:

1.0 ms = 0.001 seconds

Blur = 1.2 × 0.001

Blur = 0.0012 m

Blur = 1.2 mm

Therefore, the battery moves 1.2 mm during the exposure.

This may still produce significant blur when detecting small scratches or reading fine laser-etched characters.

5.8.3 Motion blur at 0.05 ms

Convert milliseconds into seconds:

0.05 ms = 0.00005 seconds

Blur = 1.2 × 0.00005

Blur = 0.00006 m

Blur = 0.06 mm

Therefore, the battery moves only 0.06 mm during the exposure.

This is considerably smaller than the 0.3 mm minimum defect size.

5.8.4 Motion blur comparison

Exposure time	Exposure in seconds	Motion blur at 1.2 m/s
4.0 ms	0.004	4.8 mm
1.0 ms	0.001	1.2 mm
0.05 ms	0.00005	0.06 mm

The calculation shows that reducing exposure time significantly reduces motion blur.

For this application, 0.05 ms is the most suitable of the three example exposure times, provided that sufficient light is available.

5.9 Converting Motion Blur into Pixels

Motion blur should also be evaluated in pixel units because image-processing algorithms operate on pixels.

The conversion is:

Blur in pixels = Motion blur in mm / mm per pixel

The actual mm-per-pixel value depends on the selected camera resolution and calibrated field of view.

For illustration, assume a 1920-pixel image width covering a 320 mm field of view.

Horizontal scale = 320 / 1920

= 0.1667 mm/pixel approximately

Using this scale:

4.0 ms exposure

Blur = 4.8 / 0.1667

≈ 28.8 pixels

1.0 ms exposure

Blur = 1.2 / 0.1667

≈ 7.2 pixels

0.05 ms exposure

Blur = 0.06 / 0.1667

≈ 0.36 pixels

Exposure time	Motion blur	Approximate blur at 0.1667 mm/pixel
4.0 ms	4.8 mm	28.8 pixels
1.0 ms	1.2 mm	7.2 pixels
0.05 ms	0.06 mm	0.36 pixels

This example demonstrates why short exposure times are important when the system must detect small defects.

The calculation is based on the assumed 1920-pixel horizontal resolution. A different camera resolution will produce a different pixel-blur value.

5.10 Rolling-Shutter Row-Readout Distortion

A rolling-shutter sensor reads image rows at different times.

Let:

- tr = time interval between successive row exposures or readout timing.
- N = number of image rows.
- Tskew = approximate time difference between the first and last row timing.

A simplified timing relationship is:

$T_{skew} \approx (N - 1) \times tr$

The exact timing depends on the camera sensor and its readout architecture.

During this time, the battery continues to move along the conveyor.

The approximate displacement associated with the row-timing difference is:

Displacement = Conveyor Velocity × Tskew

For the GreenCycle system:

Displacement = 1.2 × Tskew

The exact numerical distortion cannot be calculated without the sensor's row-readout timing specification.

5.10.1 Effects of rolling-shutter distortion

1. A moving cylindrical battery may appear tilted or skewed.
2. Straight edges may appear slanted.
3. Battery dimensions may be measured incorrectly.
4. Laser-etched text may appear distorted.
5. Small defects may be stretched or displaced.
6. Object boundaries may not represent one common instant.

5.10.2 Rolling-shutter distortion diagram

Figure 5.1 – Rolling-shutter acquisition

Top Row -> Captured at Time T1

Second Row -> Captured at Time T2

Third Row -> Captured at Time T3

Fourth Row -> Captured at Time T4

Bottom Row -> Captured at Time Tn

Because each row is captured at a different time, a moving object can change position during image acquisition.

5.11 Effect of 100 Hz Lighting Flicker

The assignment specifies overhead 50 Hz grid lighting that produces 100 Hz light flicker.

The period of a 100 Hz signal is:

T = 1 / f

T = 1 / 100

T = 0.01 seconds

T = 10 ms

Therefore, the light output varies with a period of approximately 10 ms.

5.11.1 Why flicker affects image acquisition

If the camera captures different rows at different times, the illumination may change during the image acquisition process.

This can produce:

- Brightness bands.
- Dark horizontal bands.
- Different brightness levels across the image.
- Inconsistent color measurements.
- Unreliable defect segmentation.

5.11.2 Effect of exposure duration

An exposure that covers a complete number of flicker cycles may reduce some brightness variation under an ideal periodic-light model.

However, the camera and lighting system must be tested together. Real LED and fluorescent lighting may have more complex waveforms, and exposure duration alone does not guarantee flicker-free images.

The selected exposure times are:

Exposure	Duration relative to 10 ms flicker period	Main consideration
4.0 ms	0.4 of a period	May capture different illumination phases
1.0 ms	0.1 of a period	Sensitive to the instantaneous light level
0.05 ms	0.005 of a period	Very short exposure; requires sufficient strobe intensity

5.11.3 Recommended lighting solution

The system should use a controlled LED lighting arrangement rather than depending only on variable ambient illumination.

A high-intensity LED strobe can be synchronized with the camera trigger. The inspection area should also be shielded from external lighting as much as practical.

This improves image consistency and reduces the effect of ambient light changes.

5.12 Photon-to-Digital Signal Chain

A digital camera converts incoming light into digital image information through a sequence of physical and electronic operations.

The main stages are:

5.12.1 Photons

Light reflected from the battery surface enters the camera lens.

The amount and wavelength distribution of incoming light depend on the illumination source and the surface material.

5.12.2 Charge integration

The image sensor contains light-sensitive pixels.

When photons reach the sensor, they generate electrical charge. The accumulated charge depends on the amount of light received and the exposure time.

5.12.3 Voltage conversion

The accumulated charge is converted into an electrical voltage signal.

This voltage represents the measured light intensity at the corresponding pixel.

5.12.4 Analog conditioning

The analog signal may undergo amplification and other conditioning operations before digitization.

This stage helps prepare the signal for conversion into digital values.

5.12.5 Analog-to-digital conversion

The analog-to-digital converter (ADC) converts the electrical signal into a numerical value.

The number of available digital levels depends on the ADC bit depth.

For example, an 8-bit output provides 256 possible values per channel.

5.12.6 Digital image output

The resulting pixel values are arranged into an image and transferred to the edge computer.

The computer can then perform image preprocessing, object detection, defect analysis and decision-making.

Figure 5.2 – Camera signal conversion chain

Light Reflected from Battery

|

v

Photons

|

v

Camera Lens

|

v

Image Sensor

|

v

Charge Integration

|

v

Charge-to-Voltage

Conversion

|

v

Analog Conditioning

|

v

Analog-to-Digital

Conversion

|

v

Digital Pixels

|

v

Edge Computer

|

5.13 CCD and CMOS Sensor Comparison

Two important image-sensor technologies are CCD and CMOS.

5.13.1 CCD – Charge-Coupled Device

A CCD sensor collects electrical charge generated by incoming light and transfers the charge through the sensor towards a readout structure.

CCD sensors have historically been used in scientific imaging and applications requiring consistent image quality.

Advantages include:

1. Good image quality in suitable implementations.
2. Historically strong performance in low-light imaging.
3. Uniform sensor characteristics in many traditional designs.

Limitations include:

1. Higher power consumption in many implementations.
2. More complex charge-transfer electronics.
3. Lower integration flexibility compared with modern CMOS designs.
4. Availability and cost may be less favourable for some industrial applications.

5.13.2 CMOS – Complementary Metal-Oxide-Semiconductor

CMOS sensors use pixel-level or column-level electronic structures to convert and read image signals.

Modern CMOS technology is widely used in industrial cameras.

Advantages include:

1. Low power consumption in many implementations.
2. High-speed readout options.
3. Integration of additional electronic functions.
4. Availability of global-shutter and rolling-shutter designs.
5. Broad selection of industrial camera formats.

Limitations include:

1. Performance depends strongly on the sensor design.
2. Rolling-shutter models may produce motion distortion.
3. Noise and dynamic range vary between sensor models.
4. Global-shutter sensors may have different sensitivity or cost characteristics.

5.13.3 Comparison table

Parameter	CCD	CMOS
Charge handling	Charge transferred across sensor	Pixel/column electronic readout
Power consumption	Often higher	Often lower
Readout speed	Depends on design	Many high-speed options
Integration	Less integrated in traditional designs	High integration possible
Shutter options	Sensor-dependent	Rolling and global shutter options
Industrial availability	Application-dependent	Widely available
Main consideration	Image quality and readout design	Speed, shutter type and integration

The sensor technology alone does not determine image quality. The actual camera must be evaluated using its sensitivity, noise, dynamic range, shutter behaviour, readout speed and interface.

5.14 Encoder Synchronization

A conveyor encoder measures the movement of the conveyor belt.

The encoder generates pulses corresponding to a specified amount of movement.

The camera system can use these pulses to trigger image acquisition at consistent conveyor positions.

For example, if the encoder produces a known number of pulses per millimetre, the control system can trigger the camera after the required displacement.

5.14.1 Benefits of encoder synchronization

1. Provides position-based triggering.
2. Reduces dependence on fixed time intervals.
3. Supports repeatable image acquisition.

4. Helps coordinate camera and strobe timing.
5. Supports accurate line-scan reconstruction.
6. Improves consistency when conveyor speed changes.

5.14.2 Application to GreenCycle Robotics

The camera should capture each battery at a known position within the field of view.

The encoder signal can trigger the global-shutter camera and LED strobe. The captured image is then processed by the edge computer.

The system must also track the battery's position between the camera and the pneumatic rejecter.

The distance from the camera centre to the piston is 0.48 m. At a conveyor speed of 1.2 m/s, the travel time is:

Time = Distance / Velocity

Time = 0.48 / 1.2

Time = 0.4 seconds

Time = 400 ms

This means the nominal travel time from the camera centre to the piston is 400 ms at the stated conveyor speed.

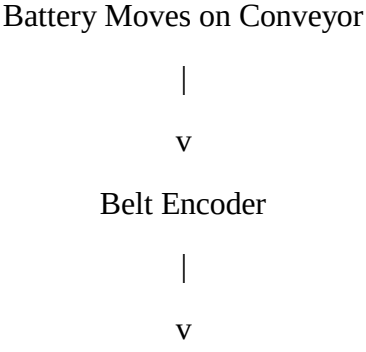
The assignment also requires a decision within 400 ms. Therefore, the system must consider the complete timing chain, including image acquisition, processing, communication, actuator response and the physical time available before the battery reaches the rejecter.

The exact actuator timing and safety margin must be measured during integration.

5.15 Recommended Acquisition Workflow

The recommended acquisition process uses a global-shutter CMOS camera with synchronized LED strobe lighting.

Figure 5.3 – Proposed physical acquisition workflow



Position Trigger

|

v

LED Strobe

|

v

Global-Shutter Camera

|

v

Image Capture

|

v

Image Transfer

|

v

Edge Computer

|

v

Defect Analysis

|

v

Sorting Decision

|

v

Pneumatic Controller

5.16 Testing and Evaluation Plan

The three acquisition methods should be evaluated using the same battery samples and conveyor conditions.

5.16.1 Test conditions

The test should include:

1. Conveyor speed of 1.2 m/s.
2. Exposure times of 4.0 ms, 1.0 ms and 0.05 ms.
3. Different ambient illumination levels.
4. Overhead lighting with the specified flicker condition.
5. Cylindrical 18650 batteries.
6. Cylindrical 21700 batteries.
7. Prismatic pouch batteries.
8. Representative scratches and casing defects.
9. Laser-etched chemistry codes.
10. Repeated image acquisition at different conveyor positions.

5.16.2 Measurements

The following values should be recorded:

Measurement	**Purpose**
Motion blur in mm	Measure physical displacement during exposure
Motion blur in pixels	Estimate image degradation
Image brightness variation	Identify lighting inconsistency
Geometric distortion	Evaluate rolling-shutter effects
Text readability	Check chemistry-code image quality
Defect visibility	Check detection of small defects
Trigger repeatability	Evaluate acquisition consistency
Processing input rate	Confirm image delivery to the edge computer

5.16.3 Result recording table

Acquisition method	**Exposure**	**Observed blur**	**Geometric distortion**	**Text readability**	**Defect visibility**

Rolling shutter	4.0 ms	To be measured	To be measured	To be tested	To be tested
Rolling shutter	1.0 ms	To be measured	To be measured	To be tested	To be tested
Rolling shutter	0.05 ms	To be measured	To be measured	To be tested	To be tested
Global shutter	4.0 ms	To be measured	To be measured	To be tested	To be tested
Global shutter	1.0 ms	To be measured	To be measured	To be tested	To be tested
Global shutter	0.05 ms	To be measured	To be measured	To be tested	To be tested
Line scan	Encoder-dependent	To be measured	To be measured	To be tested	To be tested

The table is an experimental template. The measured values should be entered after testing the selected cameras.

5.17 Engineering Recommendation

For the GreenCycle Robotics battery-sorting system, the recommended approach is a global-shutter CMOS camera combined with high-intensity LED strobe lighting and hardware encoder synchronization.

The main reasons are:

1. Global shutter reduces row-by-row motion distortion.
2. Short exposure helps reduce movement blur.
3. LED strobe lighting improves illumination control.
4. Encoder triggering supports repeatable image acquisition.
5. The arrangement is suitable for capturing moving battery surfaces.
6. It supports consistent image capture for defect detection and text recognition.

The 0.05 ms exposure example produces a calculated motion blur of 0.06 mm at 1.2 m/s. This is substantially smaller than the 0.3 mm minimum defect size.

However, the camera and lighting must provide sufficient signal at this short exposure. The final exposure and strobe settings must be validated experimentally.

A line-scan camera is a suitable alternative if continuous surface inspection and encoder-based scanning are required. A rolling-shutter camera is less preferred because moving objects and flickering illumination may introduce image distortion.

5.18 Conclusion

Physical image acquisition is a critical stage in the GreenCycle Robotics computer vision system. Camera shutter type, exposure time and lighting synchronization directly affect the quality of captured images.

A rolling-shutter camera captures different rows at different times, which can create geometric distortion when objects move quickly. A global-shutter camera captures the image at nearly the same instant and is therefore more suitable for moving-object inspection. A line-scan camera captures successive image lines and can provide continuous inspection when synchronized with conveyor displacement.

The motion-blur calculations show that exposure time has a major effect on image quality. At 1.2 m/s, exposure times of 4.0 ms, 1.0 ms and 0.05 ms produce calculated blur distances of 4.8 mm, 1.2 mm and 0.06 mm respectively.

The recommended system uses a global-shutter CMOS camera with synchronized LED strobe lighting and a belt encoder. This approach provides a practical balance between image quality, motion control and industrial integration.

5.19 Suggested Figures and Diagrams

Figure 5.4: Rolling-shutter image acquisition showing sequential row exposure.

Figure 5.5: Global-shutter image acquisition showing simultaneous exposure.

Figure 5.6: Line-scan camera capturing successive lines on a moving conveyor.

Figure 5.7: Motion-blur comparison for 4.0 ms, 1.0 ms and 0.05 ms exposure times.

Figure 5.8: Effect of 100 Hz lighting flicker on image brightness.

Figure 5.9: Photon-to-digital signal conversion chain.

Figure 5.10: CCD and CMOS sensor architecture comparison.

Figure 5.11: Encoder-triggered camera and LED strobe arrangement.

Figure 5.12: Complete camera acquisition and pneumatic sorting sequence.

TASK 6 – SPATIAL SAMPLING: RESOLUTION OPTIMIZATION VS DATA THROUGHPUT

6.1 Introduction

Spatial sampling is the process of converting a continuous real-world image into a digital image containing a finite number of pixels. The number of pixels used to represent an object determines the level of spatial detail available for image processing.

In the GreenCycle Robotics battery-sorting system, the camera must identify battery form factors, read small laser-etched chemistry codes and detect physical defects. The smallest defect that must be detected is 0.3 mm, while the laser-etched text has a stroke width of 0.4 mm.

The camera must cover a field of view of 320 mm across the conveyor and 240 mm along the conveyor. Therefore, the selected resolution must provide sufficient pixels to represent small defects without producing an excessive data-processing and storage requirement.

This task compares three camera resolutions:

1. VGA – 640 × 480 pixels.
2. Full HD – 1920 × 1080 pixels.
3. 12-megapixel – 4096 × 3000 pixels.

The comparison considers spatial resolution, millimetres per pixel, defect sampling, text readability, data bandwidth and five-day image storage.

6.2 Understanding Spatial Sampling

A real-world object contains continuous physical details. A digital camera divides the captured image into a grid of pixels.

Each pixel represents a small area of the scene. The physical size represented by one pixel depends on the camera resolution and the field of view.

Higher resolution generally provides more pixels across the same physical area.

For example, if two cameras observe the same 320 mm-wide conveyor region, the camera with more horizontal pixels represents each millimetre using more pixels.

This is important because small scratches and text strokes may disappear or become difficult to distinguish when too few pixels are available.

6.2.1 Spatial resolution

Spatial resolution describes how much physical detail can be represented in an image.

A camera with higher spatial resolution can represent smaller features using more pixels, provided that the lens, focus, sensor quality and illumination are adequate.

6.2.2 Pixel density

Pixel density can be expressed as the number of pixels representing one millimetre.

The horizontal pixel density is:

Horizontal pixel density = Image width / Field of view width

The vertical pixel density is:

Vertical pixel density = Image height / Field of view height

The corresponding physical scale is:

mm per pixel = Field of view / Number of pixels

A smaller mm-per-pixel value means that each pixel represents a smaller physical area.

6.3 Technique A – VGA Resolution (640 × 480)

6.3.1 Working principle

VGA resolution contains:

- Width = 640 pixels.
- Height = 480 pixels.
- Total pixels = 307,200 pixels.

It is a relatively low-resolution image format compared with Full HD and 12-megapixel imaging.

When the camera covers the full GreenCycle field of view, the available pixels must represent the entire conveyor inspection area.

6.3.2 Horizontal spatial sampling

The horizontal field of view is 320 mm.

Horizontal scale = 320 / 640

= 0.5 mm/pixel

Therefore, each horizontal pixel represents approximately 0.5 mm of the conveyor area.

6.3.3 Vertical spatial sampling

The vertical field of view is 240 mm.

Vertical scale = 240 / 480

= 0.5 mm/pixel

Each vertical pixel also represents approximately 0.5 mm.

6.3.4 Defect sampling

The smallest defect is 0.3 mm.

Pixels across defect = 0.3 / 0.5

= 0.6 pixels

A 0.3 mm defect is smaller than one pixel at this sampling scale.

Therefore, VGA cannot reliably represent the smallest defect as a distinct spatial feature.

The 0.4 mm laser-etched text stroke is also represented by:

0.4 / 0.5 = 0.8 pixels

This is less than one pixel and is insufficient for reliable text-stroke analysis.

6.3.5 Advantages

1. Lower image-data volume.
2. Reduced memory requirements.
3. Lower transmission bandwidth.
4. Faster processing in many situations.
5. Suitable for basic object-position detection.
6. Can be useful for simple inspection tasks where small details are not required.

6.3.6 Limitations

1. Insufficient spatial detail for 0.3 mm defects.
2. Text strokes may be represented by less than one pixel.
3. Fine scratches may disappear during sampling.
4. Small defects may become mixed with surrounding pixels.
5. Digital zoom cannot recover missing original image detail.
6. Unsuitable as the only acquisition resolution for the specified inspection requirements.

6.3.7 Suitability for GreenCycle Robotics

VGA resolution is not recommended for the main inspection camera because the smallest defect occupies less than one pixel. It may be useful for basic conveyor monitoring, but it cannot provide sufficient spatial sampling for the complete inspection requirement.

6.4 Technique B – Full HD Resolution (1920 × 1080)

6.4.1 Working principle

Full HD resolution contains:

- Width = 1920 pixels.
- Height = 1080 pixels.

- Total pixels = 2,073,600 pixels.

It provides significantly more image detail than VGA while maintaining a manageable image size for industrial processing.

6.4.2 Horizontal spatial sampling

Horizontal scale = 320 / 1920

= 0.1667 mm/pixel approximately

Therefore, one horizontal pixel represents approximately 0.1667 mm.

6.4.3 Vertical spatial sampling

Vertical scale = 240 / 1080

= 0.2222 mm/pixel approximately

One vertical pixel represents approximately 0.2222 mm.

6.4.4 Defect sampling

The smallest defect is 0.3 mm.

Horizontal sampling:

0.3 / 0.1667 ≈ 1.8 pixels

Vertical sampling:

0.3 / 0.2222 ≈ 1.35 pixels

The 0.3 mm defect is represented by fewer than two pixels in both directions.

This is better than VGA, but it is still limited for reliable detection and measurement of very small defects.

The 0.4 mm text stroke is represented by:

Horizontal:

0.4 / 0.1667 ≈ 2.4 pixels

Vertical:

0.4 / 0.2222 ≈ 1.8 pixels

The text stroke is represented by only a small number of pixels. This may be insufficient for reliable reading under motion blur, poor contrast or variable illumination.

6.4.5 Advantages

1. Provides more detail than VGA.

2. Offers a reasonable balance between image size and processing requirements.
3. Can support battery form-factor classification.
4. Produces manageable image-data volumes.
5. Can be suitable for many industrial inspection tasks.
6. Provides more information for image enhancement and feature extraction.

6.4.6 Limitations

1. The 0.3 mm defect receives fewer than two pixels in both directions.
2. Fine laser-etched strokes have limited pixel representation.
3. Digital zoom cannot restore missing spatial information.
4. Defect measurement accuracy may be affected by lens distortion and calibration.
5. High-resolution requirements may still exceed the available detail in some regions.

6.4.7 Suitability for GreenCycle Robotics

Full HD is a practical improvement over VGA and may be sufficient for general battery classification. However, the sampling calculation shows that it provides limited representation of the smallest defects and text strokes.

It should therefore be treated as a possible balanced option rather than automatically accepted as sufficient for every inspection requirement.

6.5 Technique C – 12-Megapixel Resolution (4096 × 3000)

6.5.1 Working principle

The specified 12-megapixel camera resolution is:

- Width = 4096 pixels.
- Height = 3000 pixels.
- Total pixels = 12,288,000 pixels.

It provides significantly more spatial samples than VGA and Full HD.

6.5.2 Horizontal spatial sampling

Horizontal scale = 320 / 4096

= 0.078125 mm/pixel

Each horizontal pixel represents approximately 0.0781 mm.

6.5.3 Vertical spatial sampling

Vertical scale = 240 / 3000

= 0.08 mm/pixel

Each vertical pixel represents approximately 0.08 mm.

6.5.4 Defect sampling

The smallest defect is 0.3 mm.

Horizontal sampling:

0.3 / 0.078125 = 3.84 pixels

Vertical sampling:

0.3 / 0.08 = 3.75 pixels

Therefore, the 0.3 mm defect is represented by approximately 3.75 to 3.84 pixels across its dimension, depending on its orientation.

This provides substantially more spatial information than VGA or Full HD.

6.5.5 Text-stroke sampling

The laser-etched text stroke is 0.4 mm.

Horizontal sampling:

0.4 / 0.078125 = 5.12 pixels

Vertical sampling:

0.4 / 0.08 = 5 pixels

Therefore, the text stroke is represented by approximately five pixels.

This provides a better starting point for image enhancement and optical character recognition.

However, five pixels across a stroke do not guarantee successful text recognition. Lens sharpness, focus, contrast, exposure and the character font must also be considered.

6.5.6 Advantages

1. Provides the highest spatial detail among the three options.
2. Represents 0.3 mm defects using approximately four pixels.
3. Provides approximately five pixels across a 0.4 mm text stroke.
4. Supports more detailed defect segmentation.
5. Provides additional information for measuring scratches and dents.
6. Supports image cropping and region-of-interest processing.

6.5.7 Limitations

1. Requires more memory than VGA and Full HD.
2. Produces higher image-data bandwidth.
3. Increases image-processing workload.
4. Requires suitable lens quality and focus.
5. May increase storage requirements significantly.
6. Can place additional pressure on the four-core edge computer.

6.5.8 Suitability for GreenCycle Robotics

The 12-megapixel option provides the best spatial sampling among the three specified resolutions. It is therefore the strongest candidate for detecting small defects and reading fine laser-etched text.

However, the processing and storage requirements must be evaluated carefully because the edge computer has only four CPU cores and 16 GB RAM.

6.6 Comparison of Spatial Sampling

Parameter	**VGA**	**Full HD**	**12 MP**
Image resolution	640 × 480	1920 × 1080	4096 × 3000
Total pixels	0.307 MP	2.074 MP	12.288 MP
Horizontal FOV	320 mm	320 mm	320 mm
Vertical FOV	240 mm	240 mm	240 mm
Horizontal mm/pixel	0.5000	0.1667	0.0781
Vertical mm/pixel	0.5000	0.2222	0.0800
0.3 mm defect – horizontal pixels	0.60	1.80	3.84
0.3 mm defect – vertical pixels	0.60	1.35	3.75
0.4 mm text stroke – horizontal pixels	0.80	2.40	5.12
0.4 mm text stroke – vertical pixels	0.80	1.80	5.00

Observation: The 12-megapixel option provides the best spatial representation of the smallest specified defect and text stroke. Full HD provides an intermediate level of detail, while VGA is insufficient for the smallest features.

6.7 Nyquist Sampling Principle

The Nyquist sampling principle states that a repeating signal must be sampled at more than twice its highest spatial frequency to avoid basic aliasing.

For image inspection, this principle provides a theoretical minimum sampling requirement for representing repeating spatial patterns.

However, two pixels across a feature are only a theoretical lower bound for sampling a pattern. They do not guarantee reliable defect detection, accurate edge location or OCR performance.

Real industrial inspection often requires additional pixels across a feature because of:

- Optical blur.
- Sensor noise.
- Motion blur.
- Surface reflections.
- Low contrast.
- Pixel interpolation.
- Imperfect focus.

6.7.1 Application to the 0.4 mm text stroke

A 0.4 mm stroke requires at least two samples across its width under the simplified Nyquist consideration.

The required sampling scale is:

Maximum mm/pixel = 0.4 / 2

= 0.2 mm/pixel

This is a theoretical minimum for two samples across the stroke.

Comparing the camera options:

- VGA: 0.5 mm/pixel horizontally and vertically – insufficient.
- Full HD: 0.1667 mm/pixel horizontally and 0.2222 mm/pixel vertically – horizontal sampling meets the simplified requirement, while vertical sampling does not.
- 12 MP: 0.0781 mm/pixel horizontally and 0.08 mm/pixel vertically – meets the simplified requirement in both directions.

For better OCR performance, more than two pixels across a stroke are desirable.

6.7.2 Application to the 0.3 mm defect

For a 0.3 mm defect, two samples across the feature require:

Maximum mm/pixel = 0.3 / 2

= 0.15 mm/pixel

The 12-megapixel option meets this theoretical sampling requirement in both directions.

VGA and Full HD do not meet it in both directions.

This demonstrates why the 12-megapixel option is preferred when the smallest defect must be represented clearly.

6.8 Image Data Size Calculation

Image resolution directly affects the amount of image data produced by the camera.

For an 8-bit grayscale image, each pixel requires one byte.

For an 8-bit three-channel BGR image, each pixel requires three bytes.

The uncompressed image size is:

Image size = Width × Height × Number of channels × Bytes per channel

For an 8-bit BGR image:

Image size = Width × Height × 3 × 1 byte

6.8.1 VGA image size

640 × 480 × 3

= 921,600 bytes

≈ 0.922 MB

6.8.2 Full HD image size

1920 × 1080 × 3

= 6,220,800 bytes

≈ 6.221 MB

6.8.3 12-megapixel image size

4096 × 3000 × 3

= 36,864,000 bytes

≈ 36.864 MB

These values represent uncompressed 8-bit BGR pixel data using decimal megabytes. Actual file sizes may differ because of image encoding, file headers and compression.

6.9 GigE Bandwidth Calculation

The camera data rate depends on image size and frame rate.

The basic formula is:

Data rate = Image size × Frames per second

For an uncompressed 8-bit BGR image:

Data rate = Width × Height × 3 × FPS

The following comparison uses an illustrative frame rate of 15 frames per second.

This is a calculation assumption for comparing data rates. The assignment specifies 15 cells per second, but cell throughput does not automatically mean that exactly one camera frame is required per cell. The final acquisition rate depends on camera triggering and inspection coverage.

6.9.1 VGA bandwidth at 15 FPS

921,600 × 15

= 13,824,000 bytes/second

≈ 13.824 MB/s

In bits per second:

13.824 × 8 = 110.592 Mbps

6.9.2 Full HD bandwidth at 15 FPS

6,220,800 × 15

= 93,312,000 bytes/second

≈ 93.312 MB/s

In bits per second:

93.312 × 8 = 746.496 Mbps

6.9.3 12-megapixel bandwidth at 15 FPS

36,864,000 × 15

= 552,960,000 bytes/second

≈ 552.96 MB/s

In bits per second:

552.96 × 8 = 4,423.68 Mbps

This is approximately 4.42 Gbps before accounting for transport overhead.

6.9.4 Bandwidth comparison

Resolution	Image size	Data rate at 15 FPS	Approximate bit rate
VGA	0.922 MB	13.824 MB/s	110.592 Mbps
Full HD	6.221 MB	93.312 MB/s	746.496 Mbps
12 MP	36.864 MB	552.960 MB/s	4,423.68 Mbps

A standard 1 Gigabit Ethernet link has a theoretical line rate of 1 Gbps, equivalent to 125 MB/s before protocol overhead.

Therefore, the illustrative uncompressed Full HD stream at 15 FPS is already close to the practical limits of a 1 Gigabit connection. The 12-megapixel stream at 15 FPS exceeds that link's capacity.

A higher-bandwidth interface, reduced acquisition rate, region-of-interest transmission or suitable compression may be required for the 12-megapixel option.

The camera interface must be selected based on its actual supported data rate, not only the image resolution.

6.10 Five-Day Continuous Storage Calculation

The assignment specifies a 1.5 TB SSD and a five-day visual evidence archive.

To estimate the required storage, the following assumptions are used:

- Continuous recording for five days.
- 15 frames per second.
- 8-bit BGR images.
- No image compression.
- Decimal storage units.
- No additional metadata or file-system overhead.

Five days contain:

$5 \times 24 \times 60 \times 60$

$= 432{,}000$ seconds

The storage formula is:

Total storage = Image size × FPS × Total seconds

6.10.1 VGA storage

= 921,600 × 15 × 432,000

= 5,971,968,000,000 bytes

≈ 5.97 TB

6.10.2 Full HD storage

= 6,220,800 × 15 × 432,000

= 40,310,784,000,000 bytes

≈ 40.31 TB

6.10.3 12-megapixel storage

= 36,864,000 × 15 × 432,000

= 238,878,720,000,000 bytes

≈ 238.88 TB

6.10.4 Five-day storage comparison

Resolution	Storage for five days at 15 FPS
VGA	5.97 TB
Full HD	40.31 TB
12 MP	238.88 TB

Available SSD 1.5 TB

The calculation demonstrates that continuous uncompressed recording at 15 FPS cannot fit within the specified 1.5 TB SSD for any of the three resolutions.

6.10.5 Storage requirement for practical implementation

The system should not automatically store every full-resolution frame indefinitely.

A practical evidence-storage strategy may include:

1. Store images of detected defects.
2. Store images of uncertain classification results.
3. Store images of rejected batteries.
4. Store selected normal-operation samples.

5. Compress archived images using an appropriate lossless or validated compression method.
6. Store event metadata separately.
7. Apply a retention policy to manage the five-day archive.

The system must ensure that compression does not remove important defect details.

6.11 Camera Frame Rate and Conveyor Throughput

The conveyor moves at 1.2 m/s, and the system must process 15 cells per second.

The time between successive cells at the stated throughput is:

Time per cell = 1 / 15

= 0.0667 seconds

= 66.7 ms

This means that a new cell arrives approximately every 66.7 ms on average.

The camera acquisition strategy must ensure that every battery is captured with sufficient coverage.

6.11.1 Minimum frame rate for one capture while the full battery is inside the field of view

The field of view along the conveyor is 240 mm.

The maximum battery length is 70 mm.

For the entire 70 mm battery to fit inside the 240 mm field of view, the available movement distance for its position within the field is:

Available distance = 240 - 70

= 170 mm

At 1.2 m/s:

Velocity = 1200 mm/s

The time during which the entire battery can fit inside the field of view is:

Time = 170 / 1200

= 0.1417 seconds approximately

The reciprocal is:

1 / 0.1417 ≈ 7.06 frames per second

This is an approximate minimum rate for obtaining at least one capture opportunity during that full-object visibility interval, assuming continuous suitable positioning and no other timing constraints.

It is not a guarantee of complete inspection coverage. The camera must also account for the cell's position, spacing, trigger location and required number of views.

6.11.2 Throughput-based acquisition rate

The system receives 15 cells per second.

If one frame is captured for each cell, the camera must support at least:

15 frames per second

This provides one nominal acquisition per cell.

If multiple views or repeated captures are required, the acquisition rate must be increased accordingly.

6.11.3 Frame-rate recommendation

The camera should be triggered using the conveyor encoder or a reliable cell-detection sensor.

The trigger system should ensure that:

- Every battery is captured.
- No battery is skipped.
- Images contain the complete inspection region.
- Adjacent battery images do not create unintended overlap.
- The processing system receives images at the required rate.

6.12 Region-of-Interest Processing

A region of interest (ROI) is a selected portion of an image that contains the information required for a particular operation.

Instead of processing every pixel in the complete image, the system can focus on relevant regions.

For example:

- Battery boundary ROI for form-factor classification.
- Laser-etched region for chemistry-code recognition.
- Casing surface ROI for rust and scratch detection.
- Pouch surface ROI for swelling analysis.

ROI processing can reduce computational workload.

However, cropping an image does not increase the original optical resolution. The camera must capture enough pixels in the relevant region before cropping can provide useful detail.

6.12.1 Benefits of ROI processing

1. Reduces the number of pixels processed by later algorithms.
2. Can reduce CPU workload.
3. Allows separate processing methods for different inspection regions.
4. Can reduce temporary memory requirements.
5. Helps prioritize important battery features.

6.12.2 Limitations

1. Incorrect ROI selection may remove important defects.
2. Battery-position errors can cause the ROI to miss a region.
3. The complete image may still need to be retained for evidence.
4. ROI processing cannot recover detail that was never captured.

6.13 Balancing Resolution and Processing Performance

The GreenCycle edge computer has four CPU cores and 16 GB RAM. It does not have a dedicated GPU.

Therefore, camera resolution must be selected by considering both spatial detail and processing time.

A high-resolution image provides more information, but it also increases the number of pixels that must be processed.

The following factors must be considered:

- Image acquisition rate.
- Image transfer bandwidth.
- CPU processing time.
- Memory consumption.
- Temporary image buffers.
- Image storage.
- OCR processing requirements.
- Defect-detection accuracy.

A practical implementation may use high-resolution acquisition followed by carefully selected ROI processing.

The system should measure actual processing time using the selected OpenCV algorithms on the target edge computer.

6.14 Testing and Evaluation Plan

The three camera resolutions should be tested using the same battery samples and inspection conditions.

6.14.1 Test dataset

The test images should contain:

1. 18650 cylindrical batteries.
2. 21700 cylindrical batteries.
3. Prismatic pouch batteries.
4. 0.3 mm defect samples.
5. 0.4 mm text-stroke samples.
6. Scratches of different orientations.
7. Rust spots with low contrast.
8. Different battery positions.
9. Different lighting conditions.
10. Moving battery samples at the operating conveyor speed.

6.14.2 Measurements

The following values should be recorded:

Measurement	Purpose
mm per pixel	Determine spatial sampling
Pixels across smallest defect	Evaluate defect representation
Pixels across text stroke	Evaluate OCR suitability
Image size	Estimate memory requirements
Bandwidth	Check camera interface capacity
Processing latency	Check CPU performance

Storage consumption	Evaluate archive requirements
Missed defect count	Evaluate inspection reliability
Text-reading success	Evaluate chemistry-code recognition

6.14.3 Experimental result table

Camera resolution	Defect visibility	Text readability	Processing time	Storage requirement
VGA	To be tested	To be tested	To be measured	To be measured
Full HD	To be tested	To be tested	To be measured	To be measured
12 MP	To be tested	To be tested	To be measured	To be measured

The experimental values must be obtained from the actual camera system. The calculated spatial sampling and theoretical data rates in this task are not measured camera-performance results.

6.15 Engineering Recommendation

For the GreenCycle Robotics system, the 12-megapixel camera provides the strongest spatial sampling among the three options.

It represents the 0.3 mm defect using approximately 3.75–3.84 pixels and the 0.4 mm text stroke using approximately five pixels.

This provides a better starting point for defect detection and chemistry-code recognition than VGA or Full HD.

However, the 12-megapixel option produces approximately 36.864 MB of uncompressed BGR data per image. At 15 FPS, the resulting data rate is approximately 552.96 MB/s.

Therefore, the system should use a suitable high-bandwidth camera interface, efficient ROI processing and an event-based evidence-storage strategy.

The final selection must be validated on the four-core edge computer to ensure that image acquisition, processing and sorting decisions satisfy the timing requirement.

6.16 Conclusion

Spatial sampling determines how much physical detail is represented in a digital image. It is especially important for industrial inspection tasks involving small scratches, corrosion spots and laser-etched text.

The VGA option provides only 0.5 mm per pixel and cannot adequately represent the smallest 0.3 mm defect. Full HD provides improved detail but still offers limited sampling of the smallest defect. The 12-megapixel option provides approximately 0.0781 mm per pixel horizontally and 0.08 mm per pixel vertically, making it the strongest option for detailed inspection.

However, higher resolution increases image size, bandwidth and storage requirements. The five-day uncompressed storage calculation shows that all three resolutions exceed the available 1.5 TB SSD when continuously recorded at 15 FPS.

The recommended approach is to use a sufficiently high-resolution camera, encoder-based acquisition, ROI processing and selective evidence storage. The camera and processing system must be tested together to balance spatial detail, data throughput and real-time performance.

6.17 Suggested Figures and Diagrams

Figure 6.1: Comparison of VGA, Full HD and 12-megapixel image resolution.

Figure 6.2: Pixel-grid representation of a 0.3 mm defect at the three resolutions.

Figure 6.3: Pixel-grid representation of a 0.4 mm laser-etched text stroke.

Figure 6.4: Diagram explaining spatial sampling and mm-per-pixel calculation.

Figure 6.5: Nyquist sampling concept showing two or more samples across a feature.

Figure 6.6: Comparison of uncompressed image sizes for the three resolutions.

Figure 6.7: Camera data-bandwidth comparison at 15 FPS.

Figure 6.8: Five-day image-storage comparison against the 1.5 TB SSD capacity.

Figure 6.9: Conveyor field of view showing a 70 mm battery inside the 240 mm inspection region.

Figure 6.10: Region-of-interest processing for battery casing, text and defect detection.

TASK 3 – DIGITAL REPRESENTATION: COLOR SPACE SELECTION FOR OXIDE RUST ANALYSIS

3.1 Introduction

Color-space selection is important for identifying rust on battery casings. Rust may have very low contrast, approximately 5–8 grey levels on an 8-bit scale. The system must identify these differences even when lighting changes.

3.2 Comparison of Color Spaces

A. RGB/BGR Color Space

RGB represents an image using red, green and blue channels. OpenCV normally stores color images in BGR order.

- Easy to acquire and process.
- Sensitive to changes in illumination.

- Shadows and brightness changes affect all channels.
- Direct comparison may produce inconsistent rust detection.

B. HSV Color Space

HSV represents Hue, Saturation and Value.

- Hue describes the dominant color.
- Saturation represents color intensity.
- Value represents brightness.
- Rust can be identified using suitable hue and saturation ranges.
- Value may change considerably under different lighting conditions.

*C. CIE Lab Color Space**

CIE L*a*b* separates brightness from color information.

- L* represents lightness.
- a* represents green-to-red information.
- b* represents blue-to-yellow information.
- Color differences can be measured using ΔE.
- It is useful for comparing suspected rust regions with a calibrated healthy-casing reference.

3.3 Comparison Table

Feature	**RGB/BGR**	**HSV**	**CIE L*a*b***
Brightness separation	No	Yes, Value	Yes, L*
Rust color analysis	Moderate	Good	Very good
Lighting sensitivity	High	Medium	Lower after calibration
Processing complexity	Low	Medium	Medium
Suitability	Basic	Useful	Recommended

3.4 Lighting Variation Analysis

The system must be tested under three lighting conditions.

Condition Lighting multiplier Expected effect

Normal lighting	1.0×	Baseline image
Dim lighting	0.4×	Reduced brightness and possible noise
Bright lighting	1.8×	Increased brightness and possible saturation

RGB values may change significantly with lighting. HSV separates brightness into the Value channel, while Lab separates lightness into L*. However, neither color space automatically removes illumination effects. Controlled lighting and calibration are still required.

3.5 Color Difference Calculation

The simple Euclidean color difference in Lab space is:

$$
\Delta E_{ab}=\sqrt{(L_1-L_2)^2+(a_1-a_2)^2+(b_1-b_2)^2}
$$

A larger ΔE indicates a greater color difference from the reference region. A calibrated threshold can be used to identify possible rust.

3.6 NumPy Image Representation

A color image can be represented as a three-dimensional array:

image.shape = (height, width, channels)

For a BGR image, the three channels represent blue, green and red. Pixel access uses:

image[row, column]

NumPy arrays commonly use the uint8 data type for 8-bit images.

Important: Ordinary NumPy addition on uint8 values can wrap around when the result exceeds 255. OpenCV's cv2.add() uses saturated addition and clips the result to 255.

3.7 Recommended Technique

CIE L*ab** is recommended for rust analysis because it separates lightness from color components. The system should use fixed LED lighting, reference calibration and controlled exposure to reduce false detections.

Figure 3.1: RGB, HSV and Lab color-space comparison.

TASK 4 – TOPOLOGICAL ANALYSIS: PIXEL CONNECTIVITY FOR CASING SCRATCH COUNTING

4.1 Introduction

Pixel connectivity determines whether neighbouring pixels belong to the same object. It is important when detecting scratches and counting connected defect regions.

4.2 Types of Connectivity

A. Four-Connectivity (N4)

A pixel is connected to its top, bottom, left and right neighbours.

$$
N_4(p)=\{(x-1,y),(x+1,y),(x,y-1),(x,y+1)\}
$$

It does not connect diagonal pixels.

B. Eight-Connectivity (N8)

It includes the four direct neighbours and four diagonal neighbours.

$$
N_8(p)=N_4(p)+\text{diagonal neighbours}
$$

It is useful for detecting diagonal scratches but may join objects that only touch at corners.

C. Mixed Connectivity (m-Connectivity)

Mixed connectivity combines direct and selected diagonal connections. It avoids some unwanted connections that occur in eight-connectivity.

4.3 Comparison

Feature	4-Connectivity	8-Connectivity	m-Connectivity
Direct neighbours	Yes	Yes	Yes
Diagonal neighbours	No	Yes	Conditional
Diagonal scratch detection	Weak	Good	Good
Unwanted corner joining	Low	Higher	Reduced
Complexity	Low	Low	Medium

4.4 Distance Measurements

Three common distance measurements are:

Euclidean Distance

$$
D_E=\sqrt{(x_2-x_1)^2+(y_2-y_1)^2}
$$

It measures straight-line distance.

City-Block Distance

[
D_4=|x_2-x_1|+|y_2-y_1|
]

It measures horizontal and vertical movement.

Chessboard Distance

[
D_8=\max(|x_2-x_1|,|y_2-y_1|)
]

It allows diagonal movement.

For physical scratch length, Euclidean distance is generally the most appropriate after pixel-to-millimetre calibration.

4.5 Python Implementation

```
import cv2

import numpy as np


img = cv2.imread("battery.jpg", 0)


_, binary = cv2.threshold(

    img, 100, 255, cv2.THRESH_BINARY_INV

)


count4, labels4 = cv2.connectedComponents(

    binary, connectivity=4

)


count8, labels8 = cv2.connectedComponents(

    binary, connectivity=8

)
```

print("4-connected components:", count4 - 1)

print("8-connected components:", count8 - 1)

The output can be compared to understand how connectivity changes the number of detected scratch regions.

4.6 Testing Method

Test image	4-connectivity	8-connectivity	Expected observation
Horizontal line	Connected	Connected	One scratch
Vertical line	Connected	Connected	One scratch
Diagonal line	May split	Connected	8-connectivity performs better
Checkerboard	May separate	May join	Corner connections need care
Noisy image	Depends on noise	More joining possible	Preprocessing required

The actual component counts must be measured using the test images.

4.7 Recommendation

Eight-connectivity is suitable for detecting diagonal scratches, while mixed connectivity can reduce unwanted corner connections. Euclidean distance should be used for physical measurement after calibration.

Figure 4.1: Illustration of four-, eight- and mixed-connectivity pixels.

TASK 7 – BRIGHTNESS QUANTIZATION: DYNAMIC RANGE AND DENT DETECTION

7.1 Introduction

Brightness quantization converts continuous image intensity into a limited number of digital levels. It affects the ability to identify subtle dents, rust and surface defects.

7.2 Comparison of Quantization Levels

Option	Bit depth	Intensity levels
A – Low-bit image	4-bit	16
A – Low-bit image	6-bit	64
B – Standard image	8-bit	256

C – High-bit image 12-bit 4096

A 12-bit image can represent more intensity levels than an 8-bit image. This can preserve subtle brightness differences when the camera sensor and acquisition system support them.

7.3 Quantization Step

The quantization step is:

[
\Delta=\frac{I_{max}-I_{min}}{L-1}
]

Where:

- (I_{max}) is the maximum intensity.
- (I_{min}) is the minimum intensity.
- (L) is the number of intensity levels.

For an 8-bit image, the range is 0–255. A 4-bit representation has only 16 levels, so its intensity steps are much larger.

The given rust contrast is only 5–8 grey levels on an 8-bit scale. A 4-bit image can lose this subtle difference, while 6-bit representation may preserve it inconsistently. Eight-bit and 12-bit acquisition are more appropriate.

7.4 Comparison of Techniques

A. 4-bit and 6-bit Images

- Require less storage.
- Can support simple high-speed applications.
- Lose subtle intensity information.
- May hide small dents and dark rust.

B. 8-bit Images

- Provide 256 intensity levels.
- Supported by common cameras and OpenCV.
- Offer a practical balance between speed and storage.
- May be sufficient with stable lighting.

C. 12-bit HDR Raw Images

- Provide 4096 intensity levels.

- Preserve more tonal information.
- Help analyse subtle surface differences.
- Require more storage and processing bandwidth.

Higher bit depth does not automatically guarantee higher dynamic range. Sensor noise, full-well capacity, exposure, optics and ADC quality also affect dynamic range.

7.5 False Contouring

When a smooth battery surface is converted to a low-bit image, gradual brightness changes may become visible steps. This is called false contouring or banding.

It may cause the system to:

- Identify false boundaries.
- Miss small dents.
- Confuse smooth reflections with defects.
- Produce inconsistent defect measurements.

7.6 Python Quantization Example

```
import cv2
import numpy as np

img = cv2.imread("battery.jpg", 0)

def quantize(image, bits):
    levels = 2 ** bits
    values = image.astype(np.uint16)

    q = values * (levels - 1) // 255
    restored = q * 255 // (levels - 1)

    return restored.astype(np.uint8)

img4 = quantize(img, 4)
```

```
img6 = quantize(img, 6)

img8 = quantize(img, 8)


cv2.imwrite("image_4bit.png", img4)

cv2.imwrite("image_6bit.png", img6)

cv2.imwrite("image_8bit.png", img8)
```

For a genuine 12-bit camera image stored in a 16-bit array, values may range from 0 to 4095. Converting it to 8-bit for display can be done using img12 >> 4, but this display conversion discards information.

7.7 Lighting and Quantization

Lighting	Expected problem	Recommended action
1.0×	Baseline	Maintain fixed exposure
0.4×	Noise and low contrast	Increase controlled illumination
1.8×	Saturation and lost highlights	Reduce exposure or light intensity

7.8 Image Quantization vs Model Quantization

Image quantization reduces the number of intensity levels used to represent pixels. Model quantization converts neural-network weights and activations, for example from FP32 to INT8, to reduce memory and computation.

They are different processes: one affects image acquisition precision, while the other affects model computation.

7.9 Recommendation

A 12-bit raw camera is preferred if the sensor, interface and processing hardware support it. Otherwise, an 8-bit camera with controlled LED lighting and fixed exposure is a practical alternative. Four-bit and six-bit images are not recommended for subtle rust and dent detection.

Figure 7.1: Comparison of 4-bit, 8-bit and 12-bit intensity representation.

TASK 8 – IMAGE FORMATION: GEOMETRIC PROJECTION AND PHOTOMETRIC CORRECTIONS

8.1 Introduction

Image formation describes how a real-world battery is converted into a digital image. Lens selection, camera position and illumination affect the accuracy of measurements.

8.2 Comparison of Optical Systems

A. Standard Lens with Ambient Lighting

- Low initial cost.
- Simple installation.
- Sensitive to shadows and reflections.
- Perspective distortion may affect measurements.

B. Standard Lens with Diffuse LED Dome

- Provides more uniform illumination.
- Reduces harsh reflections.
- Checkerboard calibration helps correct lens distortion.
- Suitable for controlled inspection.

C. Telecentric Lens with Diffuse Dome

- Reduces perspective-related measurement errors.
- Maintains more consistent object size over a specified depth range.
- Provides accurate dimensional measurement.
- Has higher cost and may require a more specialised installation.

8.3 Focal Length Calculation

Given:

- Sensor active width = 6.4 mm (assumed for this example).
- Object distance = 400 mm.
- Horizontal field of view = 320 mm.

Using:

$$
f=\frac{\text{Sensor width}\times\text{Object distance}}
{\text{Field of View}}
$$

$$
f=\frac{6.4\times400}{320}=8\text{ mm}
$$

Therefore, the approximate focal length is **8 mm**.

The actual focal length must be selected using the camera's true active sensor dimensions and lens specifications. The nominal 1/2-inch sensor format does not uniquely determine its active width.

8.4 Perspective Distortion

The pinhole camera projection model is:

$$
x=\frac{fX}{Z}
$$

Where:

- x is image position.
- f is focal length.
- X is the horizontal object coordinate.
- Z is object distance.

If calibration is performed at 400 mm but a surface is 30 mm closer to the camera, its distance becomes 370 mm.

The approximate scale change is:

$$
\frac{400}{370}=1.0811
$$

This represents an approximately 8.11% increase in apparent scale for the same object dimension. Therefore, measurements can be inaccurate when object height varies.

8.5 Photometric Correction

Uneven lighting creates brightness variations across the image. Flat-field correction can reduce this problem.

A simple correction is:

$$
I_{corrected}=
\frac{I_{original}-D}{F-D}\times K
$$

Where:

- D is the dark reference image.
- F is the flat-field reference image.

- K is a selected scaling constant.

Homomorphic filtering is another method. It uses the image formation model:

[
f(x,y)=i(x,y)r(x,y)
]

Taking logarithms:

[
\log f=\log i+\log r
]

This helps separate illumination from reflectance information.

8.6 Recommended System

The recommended setup is a calibrated camera with a diffuse LED dome. A telecentric lens is preferable when highly accurate dimensional measurements justify its cost. Flat-field correction and checkerboard calibration should be used to improve consistency.

Figure 8.1: Camera, lens, lighting and conveyor arrangement.

TASK 9 – SYSTEMS INTEGRATION: BALANCED FRONT-END OPTIMIZATION

9.1 Introduction

The complete system must capture images, identify battery types, detect defects and activate the pneumatic reject mechanism within 400 ms. It must also remain within the ₹1,25,000 budget.

9.2 System Comparison

Feature	**A – Budget**	**B – Balanced**	**C – Premium**
Camera	Rolling-shutter USB 1.3MP	Global-shutter GigE 2MP	4K line-scan
Lighting	Ambient	LED dome/strobe	Uniform line light
Lens	Standard	Calibrated standard	Telecentric
Image format	8-bit lossy JPEG	8-bit lossless PNG	12-bit raw
Processing	Low and mid-level	Three-level	Three-level
Cost	Low	Medium	High
Reliability	Lower	Good	Potentially very high

9.3 Processing Pipeline

The proposed processing sequence is:

1. Capture image using the camera.
2. Apply image correction and noise reduction.
3. Identify the battery boundary.
4. Extract shape and texture features.
5. Read chemistry text using OCR.
6. Classify the battery.
7. Detect swelling, dents and rust.
8. Generate a reject decision.
9. Send the signal to the pneumatic controller.
10. Store image evidence and decision information.

Block Diagram

Battery on Conveyor

|

v

Camera + LED Lighting

|

v

Image Acquisition

|

v

Preprocessing

|

v

Shape and Defect Detection

|

v

Chemistry OCR + Classification

|

v

Safety Decision

|

v

PLC / Digital Output

|

v

Pneumatic Reject Piston

|

v

Evidence Storage

9.4 Processing-Time Allocation

The following is a proposed engineering budget, not measured performance.

Processing stage	**Target time**
Image acquisition	30 ms
Preprocessing	40 ms
Segmentation and feature extraction	70 ms
Classification and OCR	100 ms
Defect decision	40 ms
Communication and output	20 ms
Safety margin	100 ms
Total	**400 ms**

Actual timing must be measured on the four-core edge PC.

9.5 Environmental Challenges

Condition	**Possible issue**	**Solution**

Storm clouds or changing light	Brightness variation	Enclosed lighting
Conveyor vibration	Blurred images	Rigid camera mounting
Metallic glare	False defects	Diffuse lighting and polarisation if needed
Dust	Lens contamination	Protective enclosure
High-speed movement	Motion blur	Global shutter and short exposure

9.6 Budget Planning

The total available budget is ₹1,25,000. A practical budget should reserve funds for:

- Camera and lens.
- LED lighting and controller.
- Edge PC and storage.
- Pneumatic cylinder and valve.
- PLC or digital I/O.
- Conveyor sensors and encoder.
- Enclosure, wiring and installation.
- Testing and contingency.

Exact industrial component prices depend on model, supplier, interface and taxes. Obtain current quotations before final procurement; do not treat estimated prices as confirmed vendor prices.

9.7 Recommendation

Option B is the preferred balance between cost and reliability. A global-shutter camera, controlled LED lighting, calibrated lens and three-level processing pipeline provide a practical foundation. Option C may provide better measurement capability but could exceed the budget and processing capacity.

The final system must be tested against the required maximum error rate of 1.5% and the 400 ms decision deadline.

Figure 9.1: Complete vision-based battery sorting system.

TASK 10 – ADAPTIVE ENGINEERING: HIGH-SPEED PLANT SCALE-UP

10.1 Introduction

The new EV contract increases the conveyor speed and throughput and introduces thermal defect detection. The system must now operate at 2.4 m/s, process 30 cells per second and make decisions within 200 ms.

It must also identify internal short circuits through subtle thermal changes that are not visible in ordinary RGB images.

10.2 Updated Operating Requirements

Parameter	Original system	New requirement
Conveyor speed	1.2 m/s	2.4 m/s
Throughput	15 cells/s	30 cells/s
Decision deadline	400 ms	200 ms
Edge PC	4 CPU cores	Same
Thermal inspection	Not required	Required
Temperature difference	Not specified	0.5°C localised spot

At 30 cells per second, the average interval between cells is:

$$
T=\frac{1}{30}=0.0333\text{ seconds}
$$

Therefore, a new cell arrives approximately every 33.3 ms.

10.3 Comparison of Scale-Up Options

A. Retain Existing System

- Lowest initial cost.
- Existing hardware and software can be reused.
- Conveyor speed doubles.
- Processing and image capture may become insufficient.
- Visible-light cameras cannot detect internal thermal defects reliably.

B. Modify Existing System

- Use a global-shutter camera.
- Add an enclosed lighting tunnel.
- Use short LED strobe exposure.
- Optimise image regions of interest.
- Apply efficient CPU processing and model quantization.

- Add a thermal sensor if its performance and interface meet the requirement.

C. Redesign the System

- Use high-speed line-scan imaging for visible inspection.
- Add a synchronised thermal imaging system.
- Use encoder-based triggering.
- Redesign acquisition and processing around the higher throughput.
- Requires greater cost and integration effort.

10.4 Motion Blur Calculation

At the new speed:

$$
v=2.4\text{ m/s}=2400\text{ mm/s}
$$

If the maximum permitted motion blur is 0.06 mm:

$$
t=\frac{\text{Permitted blur}}{\text{Conveyor speed}}
$$

$$
t=\frac{0.06}{2400}=0.000025\text{ s}
$$

$$
t=25\ \mu s
$$

Therefore, the exposure time should be approximately 25 microseconds or less to maintain this blur target.

10.5 Spatial Resolution

For the same 320 mm horizontal field of view, the spatial resolution depends on camera pixel count.

Camera width	Approximate horizontal resolution
640 pixels	0.500 mm/pixel
1920 pixels	0.167 mm/pixel
4096 pixels	0.078 mm/pixel

Increasing conveyor speed does not itself change mm/pixel. It increases the need for faster exposure, triggering and image processing.

10.6 Thermal Inspection

Visible-light cameras cannot directly identify internal short circuits. Thermal imaging is required to observe temperature differences.

The specified 0.5°C thermal difference must be verified against the camera's NETD, calibration accuracy, spatial resolution and operating conditions. NETD and absolute temperature accuracy are not the same.

A further engineering issue is that many uncooled microbolometer cameras operate in the LWIR band, while MWIR systems commonly use different detector technologies and may require cooling. The proposed MWIR camera must therefore be confirmed with a suitable supplier rather than assuming that any uncooled thermal camera will satisfy the requirement.

10.7 Data and Processing Challenges

The increased speed creates:

- More images per second.
- Higher camera-interface bandwidth.
- Greater CPU demand.
- Less time for OCR and classification.
- Greater storage requirements.

Possible optimisations include:

- Processing only battery regions of interest.
- Reducing unnecessary image copies.
- Using efficient OpenCV operations.
- Applying calibrated INT8 inference where supported.
- Storing defect images and selected evidence instead of every full-resolution frame.

10.8 Few-Shot Learning

Few-shot learning can help the system recognise new thermal defect patterns using a small number of labelled examples.

A pretrained vision model may be adapted using a limited set of thermal images. However, visible-image models cannot automatically be assumed to work on MWIR images. Differences in sensor type, temperature range and image appearance create domain-shift risks.

The system must validate new classes using independent test images and preserve traceable model versions.

10.9 Final Recommendation

Option C is the strongest long-term architecture because the new contract introduces a different sensing requirement and doubles conveyor speed. A visible line-scan system combined with a suitable thermal imaging system provides a better foundation for scale-up.

However, the thermal detector, processing time, camera interface and budget must be verified before implementation. The existing four-core PC may become a major limitation, so actual performance testing is essential.

Figure 10.1: Proposed high-speed visible and thermal inspection system.

CONCLUSION

The GreenCycle Robotics battery sorting system demonstrates how image processing, computer vision and machine vision can improve industrial safety and sorting accuracy.

The assignment compared different approaches to image representation, pixel connectivity, camera acquisition, spatial resolution, brightness quantization and geometric image formation. It also examined how these methods can be integrated into a complete automated sorting system.

A global-shutter camera, controlled LED lighting, suitable image resolution and reliable classification pipeline are recommended for the original system. The system must be validated against the 400 ms decision deadline and the maximum permitted error rate of 1.5%.

For the future high-speed plant, additional thermal sensing and a redesigned acquisition architecture are required. Overall, computer vision can reduce manual inspection effort, improve traceability and support safer battery recycling.

REFERENCES

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th Edition, Pearson, 2018.
2. S. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd Edition, Springer, 2022.
3. OpenCV Documentation, Image Processing and Computer Vision Library. https://docs.opencv.org/
4. NumPy Documentation, Array Processing and Numerical Computing. https://numpy.org/doc/
5. R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd Edition, Cambridge University Press, 2004.
6. ISO 12233, Photography – Electronic Still Picture Imaging – Resolution and Spatial Frequency Responses.

7. ISO 9283, Manipulating Industrial Robots – Performance Criteria and Related Test Methods.
8. Python Software Foundation, Python Documentation. https://docs.python.org/
9. International Electrotechnical Commission, IEC 62676, Video Surveillance Systems for Use in Security Applications.
10. National Institute of Standards and Technology, *Guide to Industrial Control Systems Security*, NIST SP 800-82.

APPENDIX A – IMPORTANT FORMULAS

Concept	Formula
Quantization levels	(L=2^b)
Quantization step	(\Delta=(I_{max}-I_{min})/(L-1))
Euclidean distance	(D_E=\sqrt{(x_2-x_1)^2+(y_2-y_1)^2})
City-block distance	(D_4= x_2-x_1 + y_2-y_1)
Chessboard distance	(D_8=\max(x_2-x_1 , y_2-y_1))
Camera focal length	(f=(Sensor\ width\times Distance)/FOV)
Motion blur	(B=v\times t)
Cell interval	(T=1/\text{Throughput})
Cohen's Kappa	(\kappa=(P_o-P_e)/(1-P_e))

APPENDIX B – PROPOSED TESTING CHECKLIST

1. Test cylindrical 18650 and 21700 batteries.
2. Test prismatic pouch batteries.
3. Test all four chemistry labels.
4. Test rust, dents and swelling.
5. Test different ambient lighting conditions.
6. Test moving batteries at 1.2 m/s.
7. Measure image acquisition and processing latency.
8. Test pneumatic reject timing.

9. Record false-positive and false-negative results.
10. Verify the maximum 1.5% error requirement.

APPENDIX C – GLOSSARY

- **ADC:** Analog-to-Digital Converter.
- **BGR:** Blue, Green and Red color representation.
- **CMOS:** Complementary Metal-Oxide-Semiconductor image sensor.
- **CV:** Computer Vision.
- **FOV:** Field of View.
- **HDR:** High Dynamic Range.
- **HSV:** Hue, Saturation and Value.
- **OCR:** Optical Character Recognition.
- **ROI:** Region of Interest.
- **SVM:** Support Vector Machine.
- **GigE:** Gigabit Ethernet.
- **MWIR:** Mid-Wave Infrared.
- **NETD:** Noise Equivalent Temperature Difference.
- **PLC:** Programmable Logic Controller.